Full adder & Full Subtractor

CENG 2112.01 Lab for Digital Circuits

Submitted by

David Dulaney

2344357

Computer Science

University of Houston-Clear Lake

Houston, Texas 77058

Mar 3,2026

Contents

Abstract 2

Introduction 2

Requirements 3

Prelab 4

Implementation 7

Design Code/Design Diagrams 7

Schematics 13

Testbench 14

Waveform/Results 23

Conclusion 45

Abstract

A full adder is a combinational circuit that requires three inputs and will produce two outputs. The first two inputs are normal inputs (A and B), the third input is a carry (Cin). The normal output is the SUM, which is the normal sum of A and B. The other output (Cout) is a carry output which transfers over if a 0 or a 1 is being carried over. For example 1 + 1 + 1 will be a SUM of 1 and have a Cout as 1. A full subtractor is similar to a full adder but instead of a cout and sum, the outputs are diff and bout which means when multiple inputs happen, for example 1 0 1 and they get subtracted. The diff would output 0 and the bout would also be 0 since a 1 didn't have to be carried to execute the subtraction.

Introduction

A full adder (#7483) is a device that when given three inputs, it will produce two outputs, with one being the sum of the inputs and the other being the carry value of the inputs (indicated by a 0 or 1). The purpose of the full adder is to perform binary addition and carrying the output

through multiple adders to produce a carry and sum output. This is used in calculators to be able to do complex calculations through binary addition. In order to understand how full adders work, minterm or maxterm expansion is used to get the Boolean expression of the full adder, it then can be simplified to conclude the most efficient circuit. A full subtractor (#74283) is a device when given three inputs, it will produce two outputs, with one being diff and the other being bout which is the bit needed to be carried to execute the subtraction (indicated by a 0 or 1). In order to understand how full subtractors work, minterm or maxterm expansion is used to get the Boolean expression of the full subtractor, it then can be simplified to conclude the most efficient circuit.

Requirements

The requirement for this lab is to simulate a full adder and full subtractor in two different equations, one being a normal minterm expansion and the other using an xor to show that their outputs are the same in Multisim with both systems receiving the same inputs. Then we used a logic analyzer within Multisim to check the waveforms of different outputs (1 and 0). Then we created these circuits on a bread board to produce and confirm our simulation is correct on Multisim. We also coded a schematic in Vivado using VHDL as well as to have a testbench of the waveforms to match the results from Multisim.

Prelab

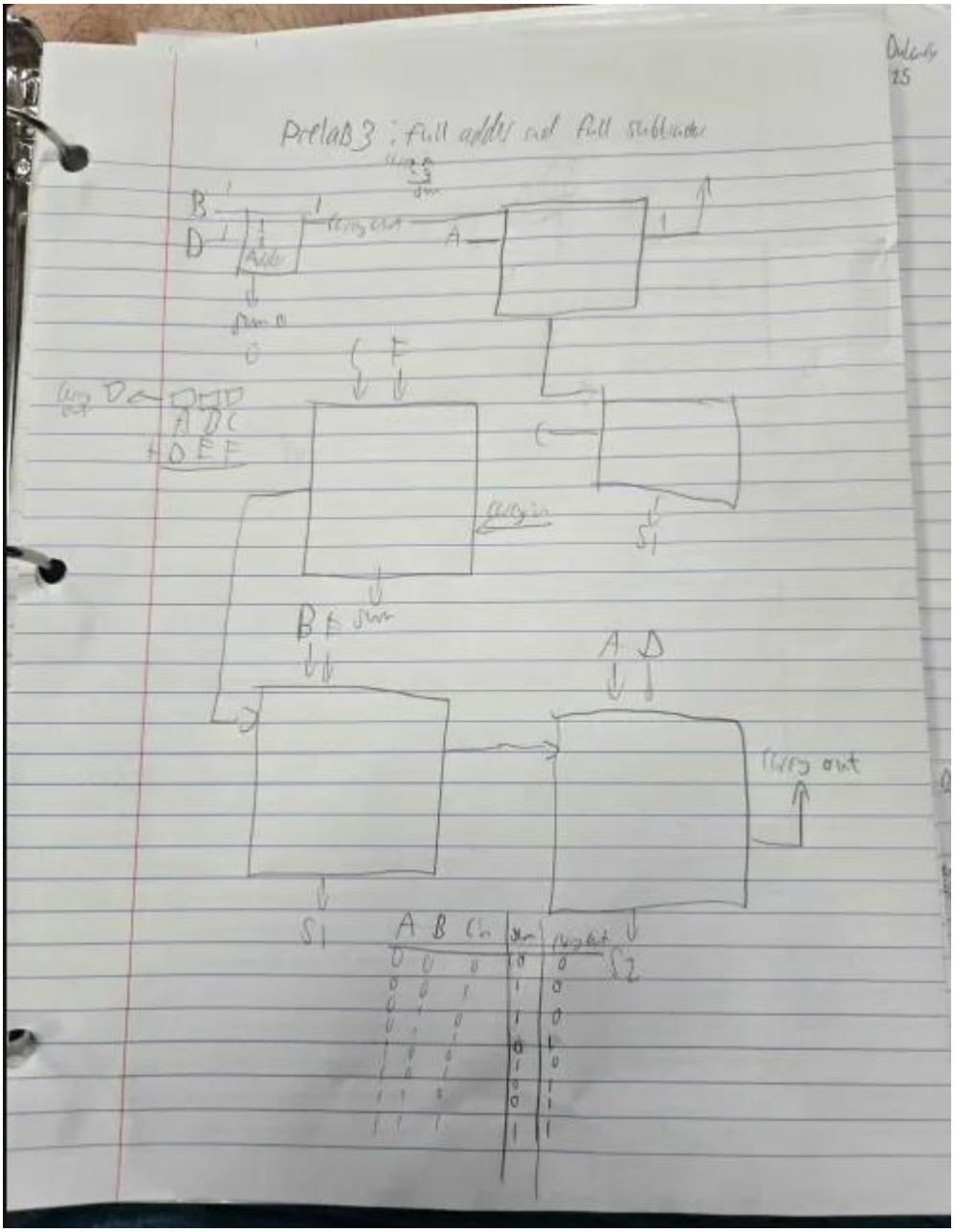

figure 1.1: prelab for full adder and truth table with sum and carry out

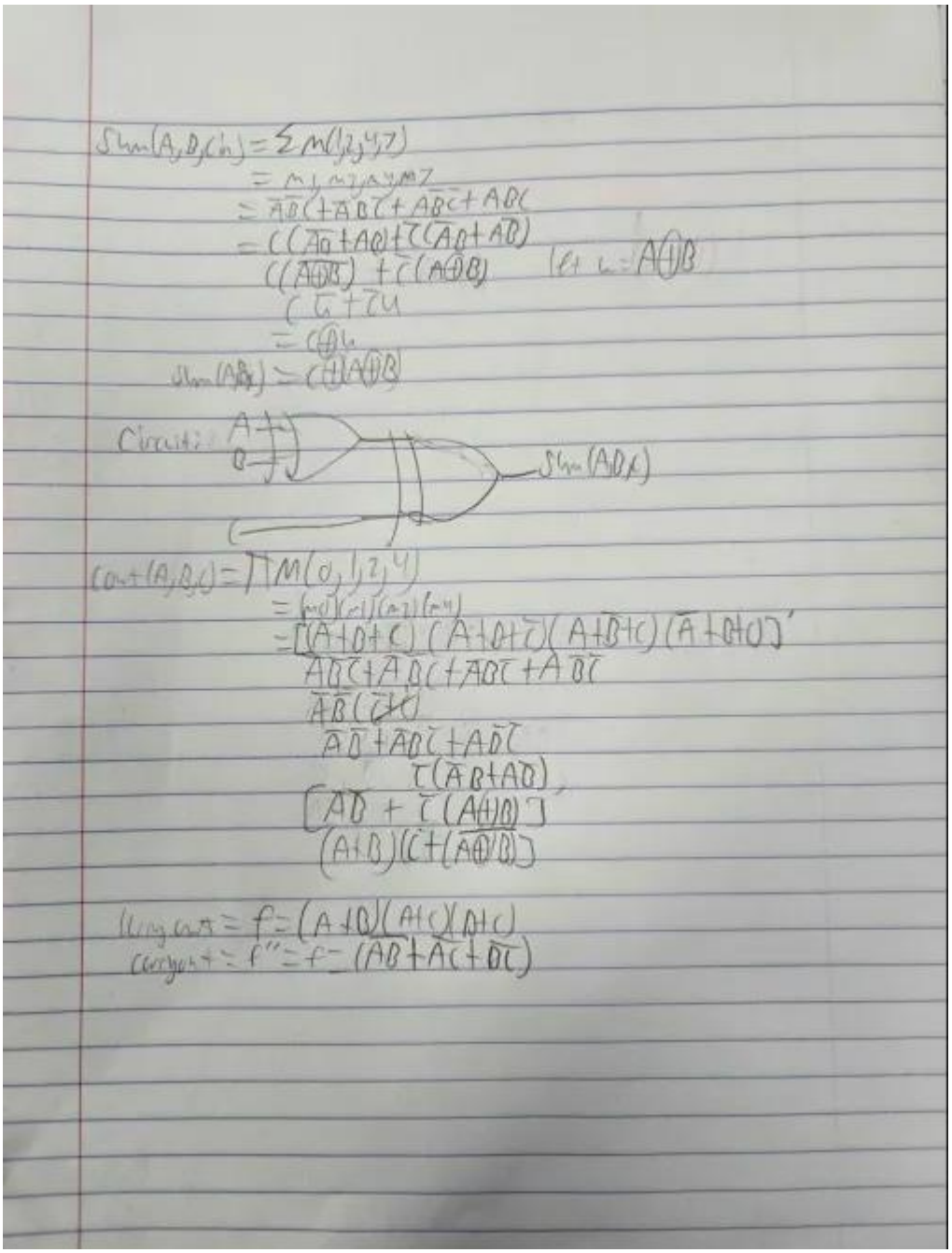

figure 1.2: minterm expansion of a full adder with the simplified version using xor with the circuit, same process for maxterm expansion minus the drawing.

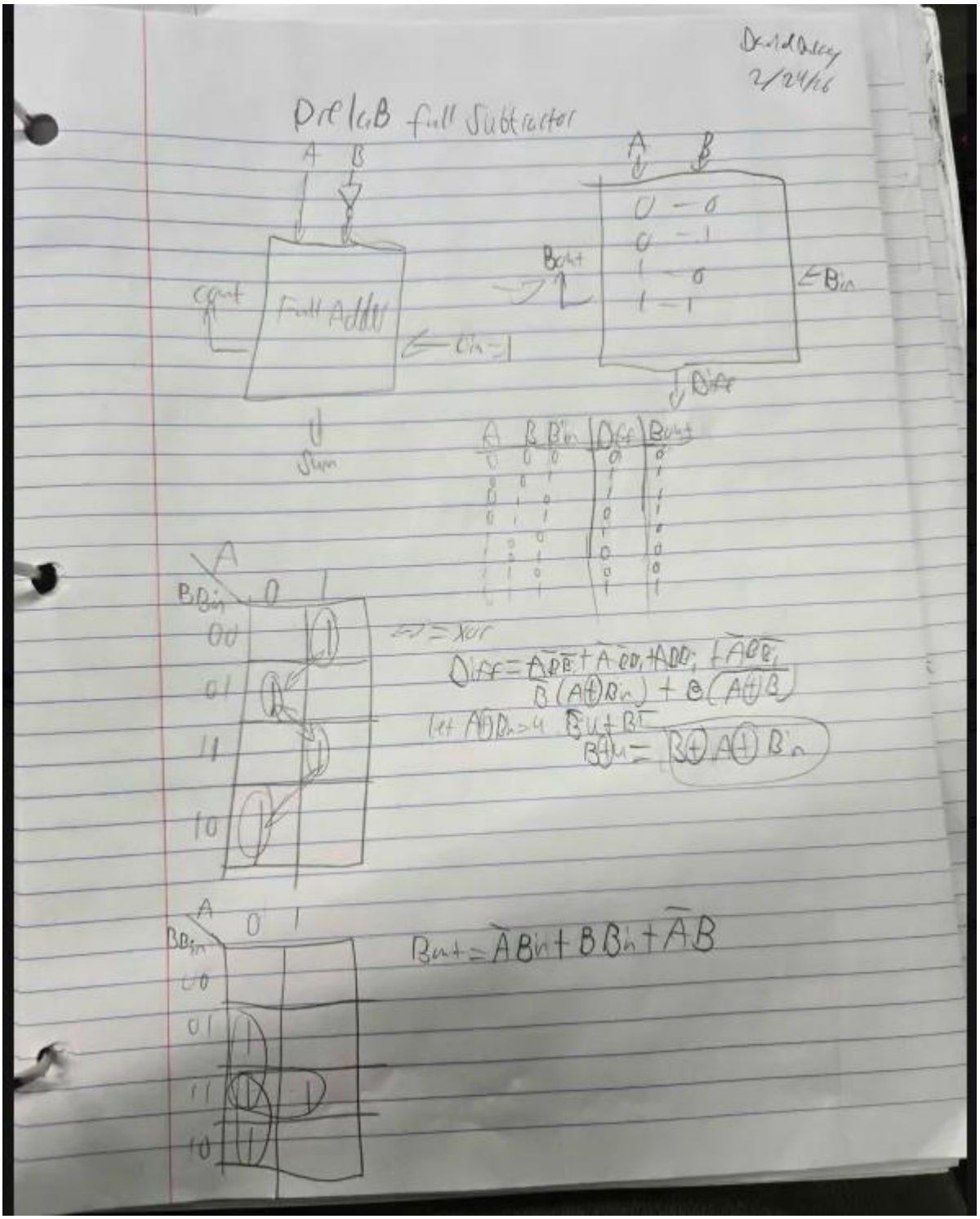

Figure 1.3 prelab for full subtractor and truth table with diff and Bout

Implementation

The first step in implementation was to build a schematic with a word generator and a logic analyzer with a full adder circuit based on eight combinations of 0s and 1s to produce the same outputs. Then we simulated these combinations in VHDL to see if we get the same outcome as our schematics.

Design Code/Design Diagrams

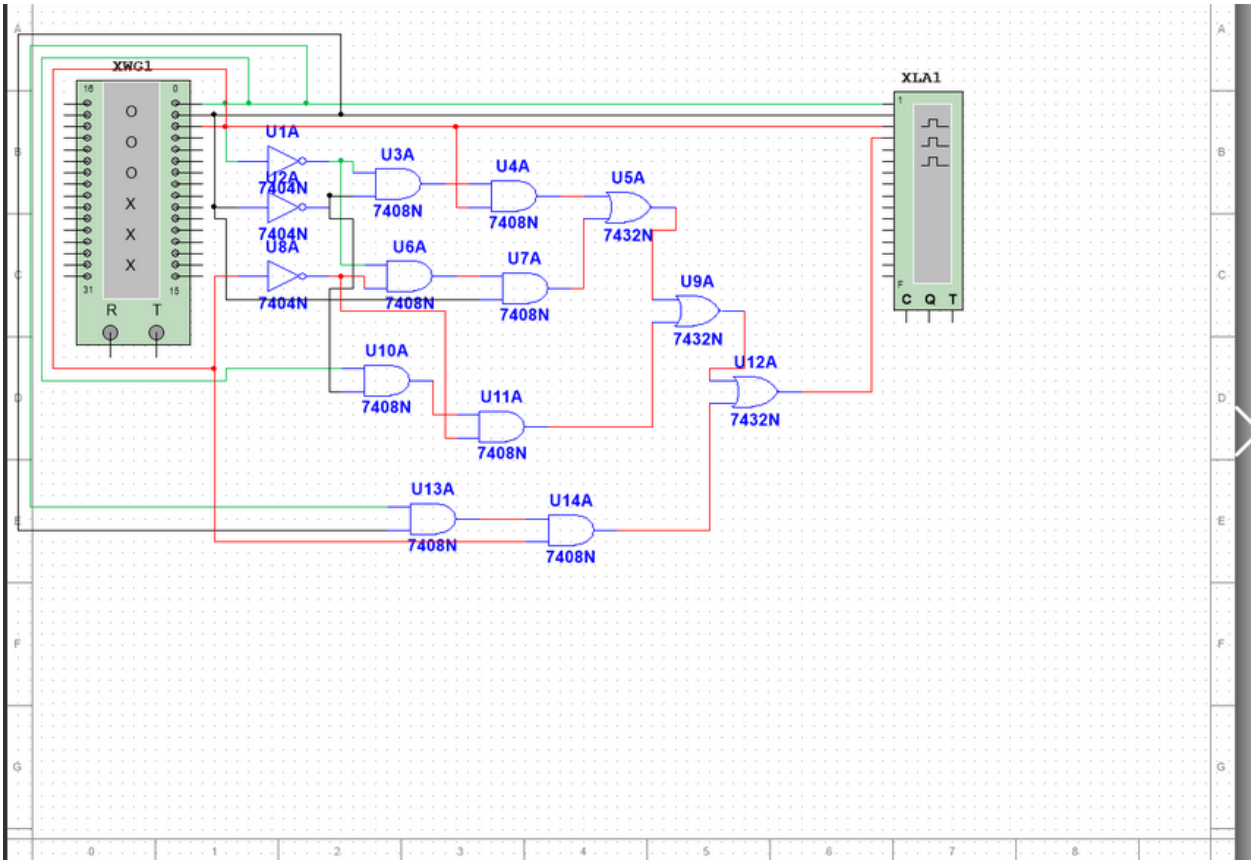

Figure 2.1: Multisim for minterm expansion for full adder

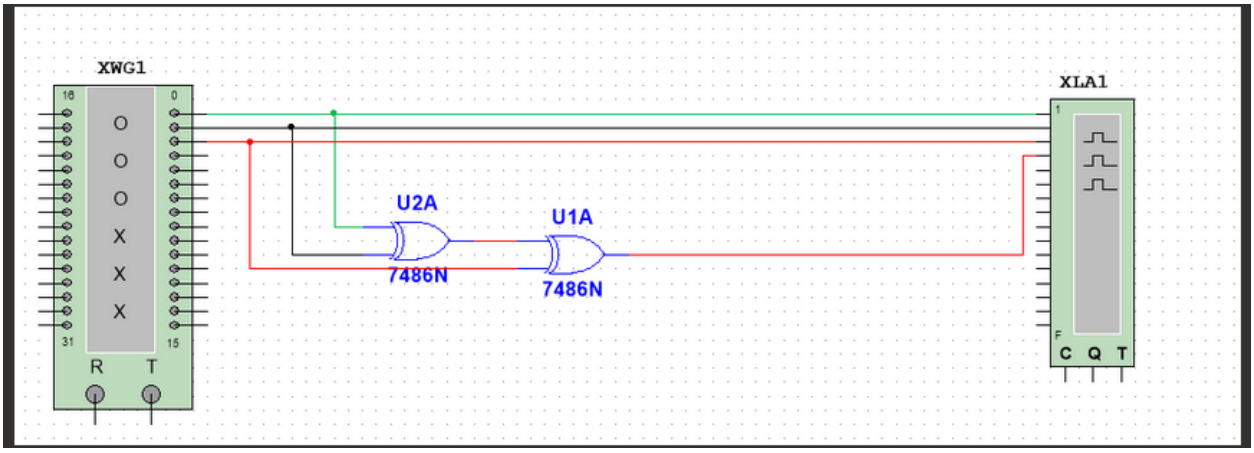

Figure 2.2: Multisim for xor for full adder

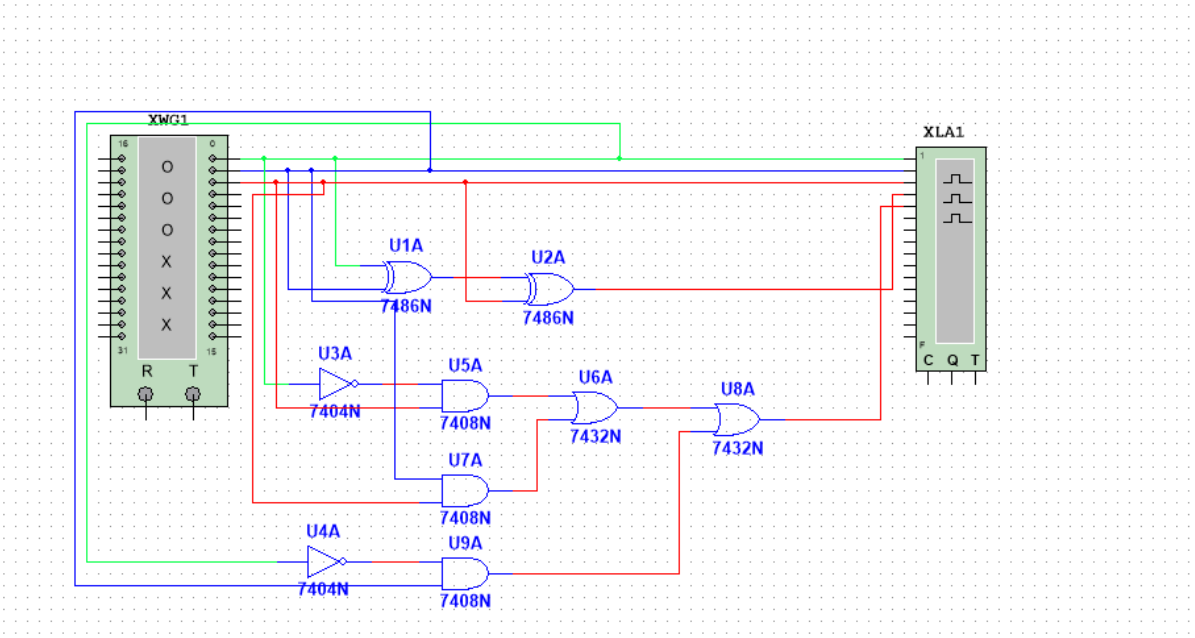

Figure 2.3: Multisim for minterm expansion and xor equations of full subtractor

We concluded with these designs because even though they are different expressions, they both produce the same output as the prelab. Figure 2.1 is the circuit for the minterm expansion while figure 2.2 is the circuit of the xor circuit. Similar to De morgan's theorem on an expression, the minterm expansion and the simplified version of that expansion (which simplified to xor) given the same inputs, will produce the same outputs. Also in figure 2.3 is the figure of the xor and minterm expansion of a full subtractor which will also produce the same outputs for that full subtractor. This is shown in figures 6.1-6.14.

DM code: for full adder

-- Company:

```
-- Engineer:
--
-- Create Date: 02/17/2026 03:27:39 PM
-- Design Name:
-- Module Name: project3 - Behavioral
-- Project Name:
-- Target Devices:
-- Tool Versions:
-- Description:
--
-- Dependencies:
--
-- Revision:
-- Revision 0.01 - File Created
-- Additional Comments:
--
----------------------------------------------------------------------------------


library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

-- Uncomment the following library declaration if using
-- arithmetic functions with Signed or Unsigned values
--use IEEE.NUMERIC_STD.ALL;
```

```
-- Uncomment the following library declaration if instantiating
-- any Xilinx leaf cells in this code.
--library UNISIM;
--use UNISIM.VComponents.all;

--instantiating the ports with input and output logic
entity project3 is
    Port ( A : in STD_LOGIC;
           B : in STD_LOGIC;
           C : in STD_LOGIC;
           SUM : out STD_LOGIC;
           COUT : out STD_LOGIC);
end project3;

architecture Behavioral of project3 is

begin
--inserted logic box for minterm in SOP and xor forms
SUM<=(A xor B) xor C;
cout<= ((A xor B) and C) or (A and B);



end Behavioral;
```

Dm code for full subtractor:

```
-- Company:
-- Engineer:
--
-- Create Date: 02/24/2026 02:43:25 PM
-- Design Name:
-- Module Name: project_5 - Behavioral
-- Project Name:
-- Target Devices:
-- Tool Versions:
-- Description:
--
-- Dependencies:
--
-- Revision:
-- Revision 0.01 - File Created
-- Additional Comments:
--
----------------------------------------------------------------------------------


library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

-- Uncomment the following library declaration if using
-- arithmetic functions with Signed or Unsigned values
--use IEEE.NUMERIC_STD.ALL;
```

```
-- Uncomment the following library declaration if instantiating
-- any Xilinx leaf cells in this code.
--library UNISIM;
--use UNISIM.VComponents.all;
--instantiating the ports with input and output logic
entity project_5 is
    Port ( A : in STD_LOGIC;
           B : in STD_LOGIC;
           Bin : in STD_LOGIC;
           Diff : out STD_LOGIC;
           Bout : out STD_LOGIC);
end project_5;

architecture Behavioral of project_5 is

begin
--inserted logic box for diff and bout kmaps equations
Diff<=A xor B xor Bin;
Bout<=(not A and Bin) or (B and Bin) or (not A and B);



end Behavioral;
```

Schematics/Implemented design

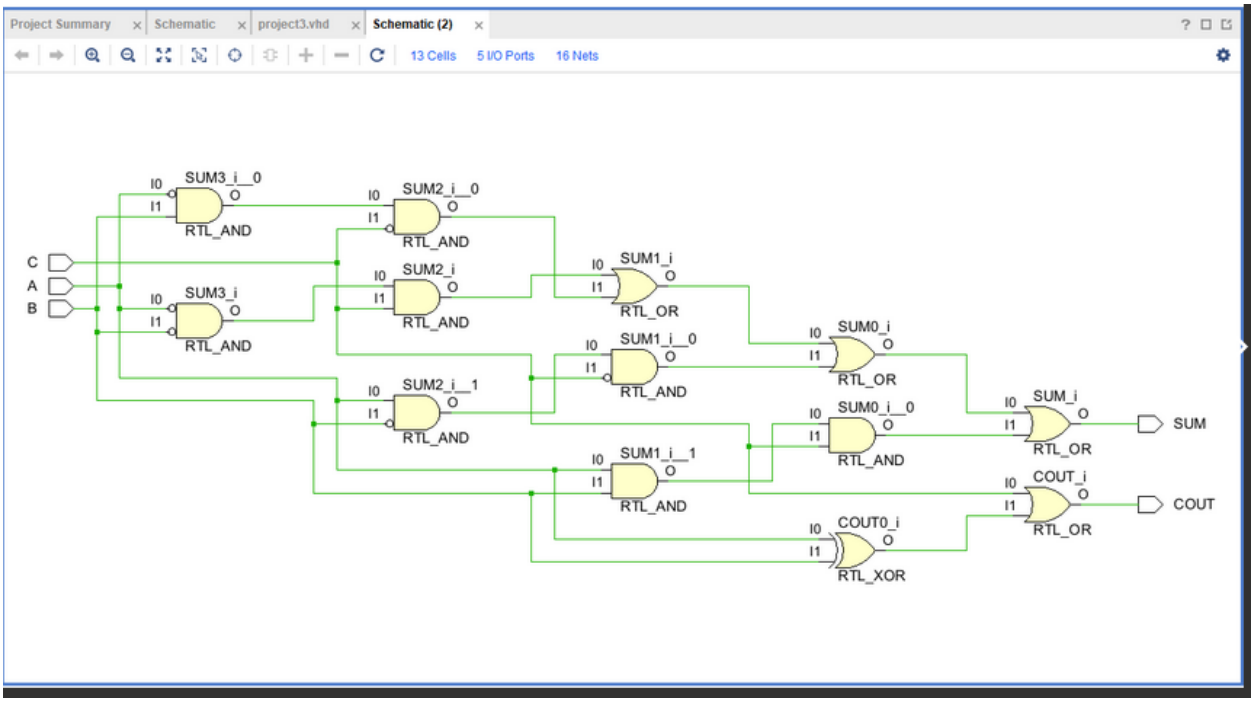

Figure 3.1: Schematic of the minterm expansion along with the simplified equation using xor for full adder

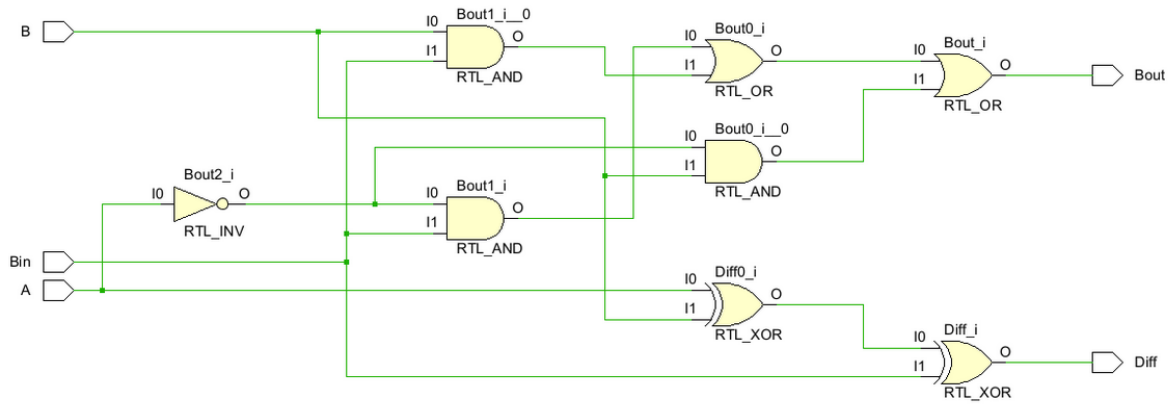

Figure 3.2: Schematic of the minterm expansion along with the simplified equation using xor for full subtractor

Testbench

TB code for full adder:

```
----------------------------------------------------------------------------------
-- Company:
-- Engineer:
--
-- Create Date: 02/17/2026 03:41:51 PM
-- Design Name:
-- Module Name: TB - Behavioral
-- Project Name:
-- Target Devices:
-- Tool Versions:
-- Description:
--
-- Dependencies:
--
-- Revision:
-- Revision 0.01 - File Created
-- Additional Comments:
--
----------------------------------------------------------------------------------



library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
```

```
-- Uncomment the following library declaration if using
-- arithmetic functions with Signed or Unsigned values
--use IEEE.NUMERIC_STD.ALL;

-- Uncomment the following library declaration if instantiating
-- any Xilinx leaf cells in this code.
--library UNISIM;
--use UNISIM.VComponents.all;

entity TB is
--  Port ( );
end TB;

architecture Behavioral of TB is
--place chip on board
component dm is
Port ( A : in STD_LOGIC;
       B : in STD_LOGIC;
       C : in STD_LOGIC;
       SUM : out STD_LOGIC;
       COUT : out STD_LOGIC);
end component;
signal A,B,C:std_logic;
signal SUM,COUT:std_logic; --instantiating wires for each port.
begin
uut: dm PORT MAP( A,B,C,SUM,COUT);
```

```
--giving 8 combinations of inputs to each set of inputs
process
begin
a<='0';
b<='0';
c<='0';
wait for 10ns;
a<='1';
b<='0';
c<='0';
wait for 10ns;
a<='0';
b<='1';
c<='0';
wait for 10ns;
a<='1';
b<='1';
c<='0';
wait for 10ns;
a<='0';
b<='0';
c<='1';
wait for 10ns;
a<='1';
b<='0';
c<='1';
```

```
wait for 10ns;
a<='0';
b<='1';
c<='1';
wait for 10ns;
a<='1';
b<='1';
c<='1';
wait for 10ns;
end process;
end Behavioral;
```

TB code for full subtractor:

```
----------------------------------------------------------------------------------
-- Company:
-- Engineer:
--
-- Create Date: 02/24/2026 02:48:22 PM
-- Design Name:
-- Module Name: TB5 - Behavioral
-- Project Name:
-- Target Devices:
-- Tool Versions:
-- Description:
--
-- Dependencies:
--
```

```
-- Revision:
-- Revision 0.01 - File Created
-- Additional Comments:
--
----------------------------------------------------------------------------------


library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

-- Uncomment the following library declaration if using
-- arithmetic functions with Signed or Unsigned values
--use IEEE.NUMERIC_STD.ALL;

-- Uncomment the following library declaration if instantiating
-- any Xilinx leaf cells in this code.
--library UNISIM;
--use UNISIM.VComponents.all;

entity TB5 is
--  Port ( );
end TB5;

architecture Behavioral of TB5 is
--place chip on board
component dm is
```

```
Port     (A :in STD_LOGIC;
         B : in STD_LOGIC;
         Bin : in STD_LOGIC;
         Diff : out STD_LOGIC;
         Bout : out STD_LOGIC);
end component;
signal A,B,Bin:std_logic;
signal Diff,Bout:std_logic;--instantiating wires for each port.
begin
uut: dm port map(A,B,Bin,Diff,Bout);
--giving 8 combinations of inputs to each set of inputs
process
begin
a<='0';
b<='0';
Bin<='0';
wait for 10ns;
a<='0';
b<='0';
Bin<='1';
wait for 10ns;
a<='0';
b<='1';
Bin<='0';
wait for 10ns;
a<='0';
```

```
b<='1';
Bin<='1';
wait for 10ns;
a<='1';
b<='0';
Bin<='0';
wait for 10ns;
a<='1';
b<='0';
Bin<='1';
wait for 10ns;
a<='1';
b<='1';
Bin<='0';
wait for 10ns;
a<='1';
b<='1';
Bin<='1';
wait for 10ns;
end process;
end Behavioral;
```

ARDUINO BOARD code for full adder:

```
-- This file is a general .xdc for the Basys3 rev B board

-- instantiating the switches to ports
```

```
set_property PACKAGE_PIN V17 [get_ports {A}]
   set_property IOSTANDARD LVCMOS33 [get_ports {A}]
set_property PACKAGE_PIN V16 [get_ports {B}]
   set_property IOSTANDARD LVCMOS33 [get_ports {B}]
set_property PACKAGE_PIN W16 [get_ports {C}]
   set_property IOSTANDARD LVCMOS33 [get_ports {C}]

--instantiating the leds to ports
set_property PACKAGE_PIN U16 [get_ports {SUM}]
   set_property IOSTANDARD LVCMOS33 [get_ports {SUM}]
set_property PACKAGE_PIN E19 [get_ports {COUT}]
   set_property IOSTANDARD LVCMOS33 [get_ports {COUT}]
```

ARDUINO BOARD code for full subtractor:

```
-- This file is a general .xdc for the Basys3 rev B board

-- instantiating the switches to ports
set_property PACKAGE_PIN V17 [get_ports {A}]
   set_property IOSTANDARD LVCMOS33 [get_ports {A}]
set_property PACKAGE_PIN V16 [get_ports {B}]
   set_property IOSTANDARD LVCMOS33 [get_ports {B}]
set_property PACKAGE_PIN W16 [get_ports {Bin}]
   set_property IOSTANDARD LVCMOS33 [get_ports {Bin}]

--instantiating the leds to ports
set_property PACKAGE_PIN U16 [get_ports {Diff}]
   set_property IOSTANDARD LVCMOS33 [get_ports {Diff}]
```

set_property PACKAGE_PIN E19 [get_ports {Bout}]

set_property IOSTANDARD LVCMOS33 [get_ports {Bout}]

Implemented design for full adder:

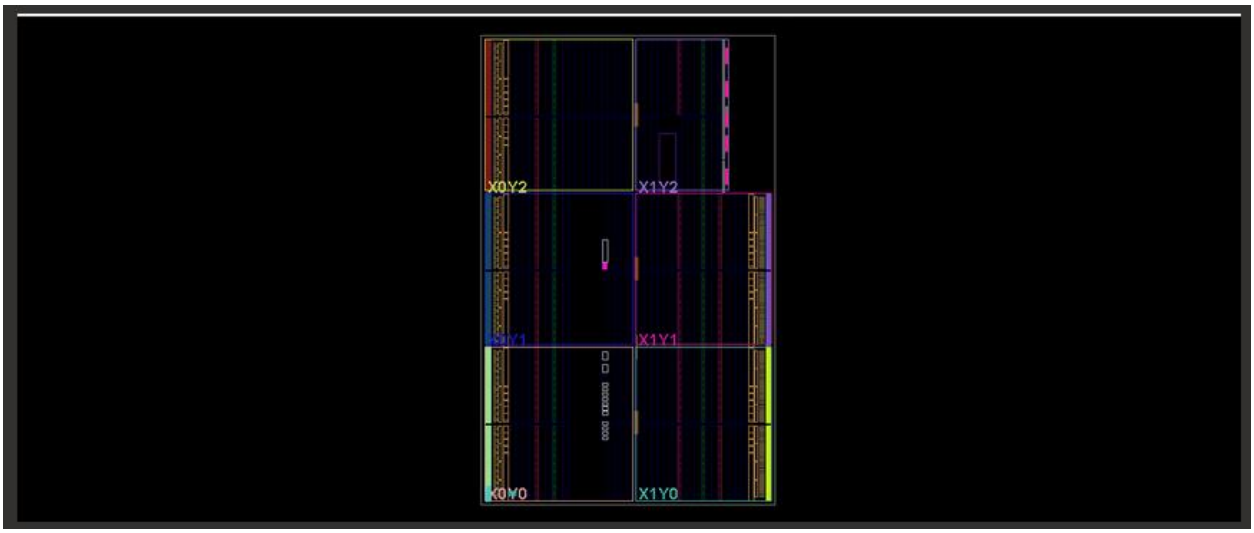

Figure 4.1: implemented design of the minterm expansion and simplified expansion using xor for full adder

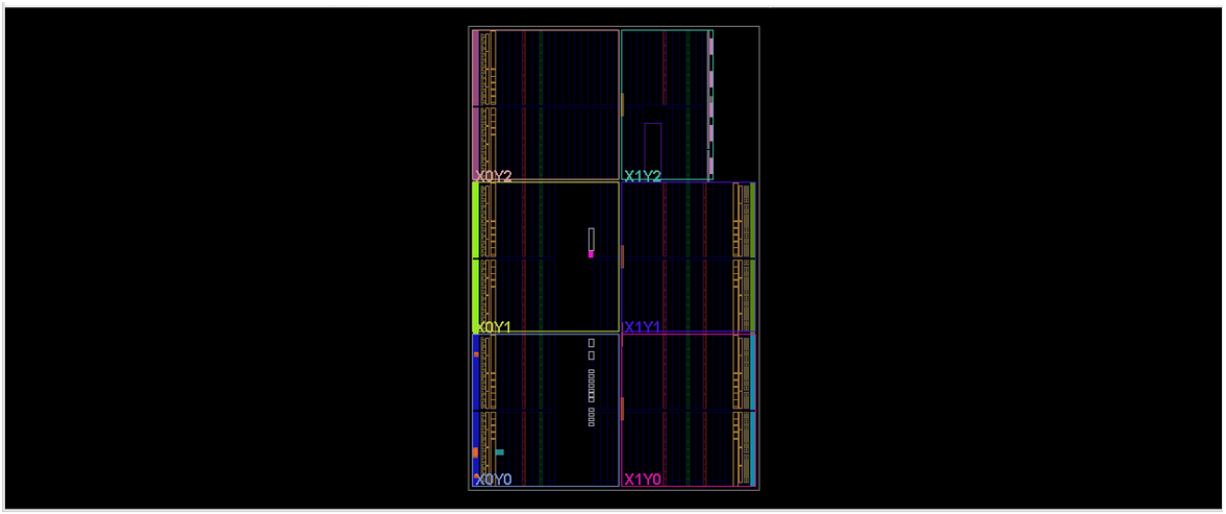

Figure 4.2: implemented design of the minterm expansion and simplified expansion using xor for full subtractor

Waveform/Results

The waveforms in figures 4.1-4.2 represent The minterm expansion and the simplified expression using xor to show the same outputs in both the full adder and full subtractor respectively. In figure 4.1-4.2, the red line shows the voltage when both inputs are fluctuating between four combinations of 0s and 1s. Where green, black, and red have the same inputs for full adders while in figure 4.3 the green and blue and top red line have the same inputs for a full subtractor. The output is based on the bottom red line, showing four combinations of 1s and 0s, the output is 1 for both waveforms. This is also identical to the behavioral simulation of both expressions shown in figure 6.1 and 6.2.

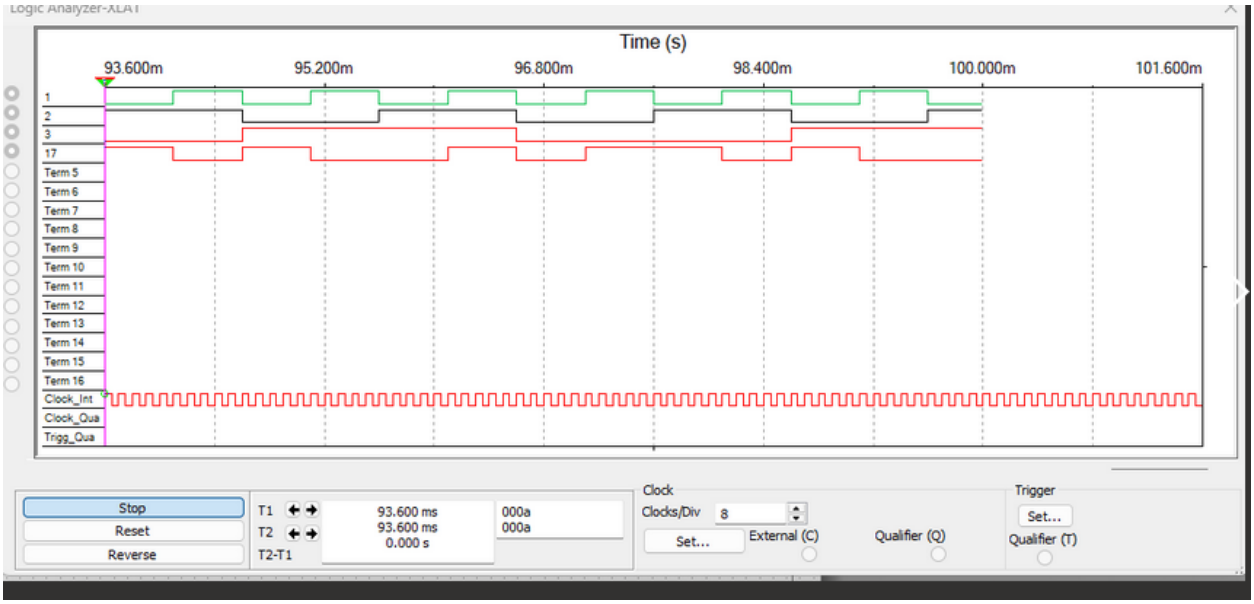

Figure 4.1: Waveform of the minterm expansion with three inputs for full adder

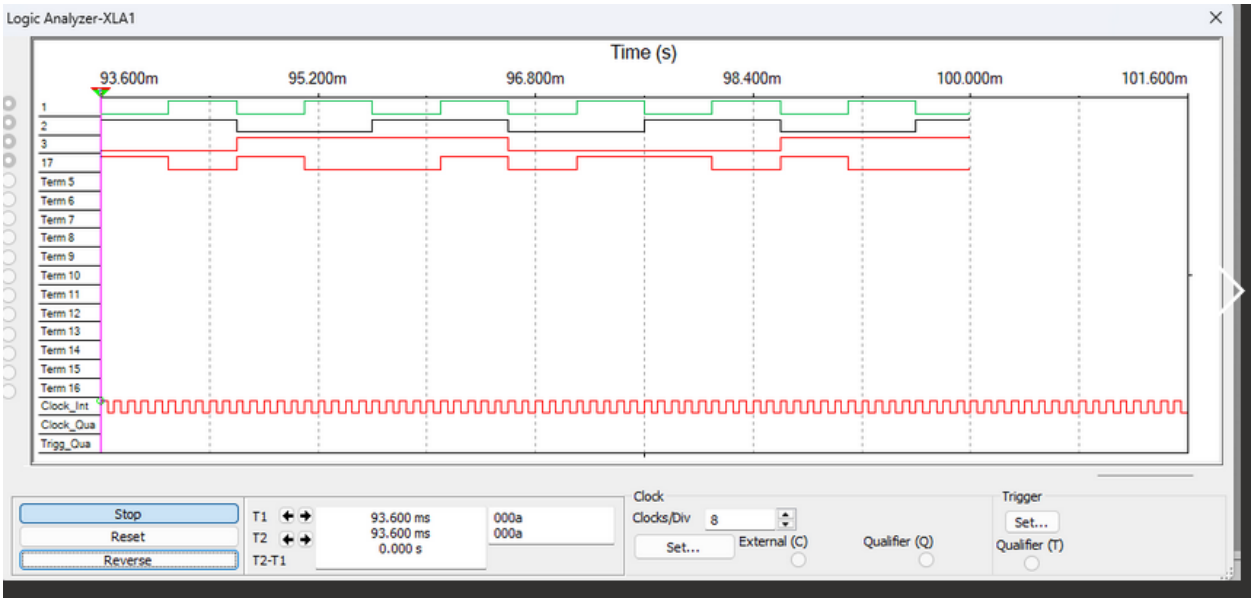

Figure 4.2: Waveform of the simplified expression using xor for full adder

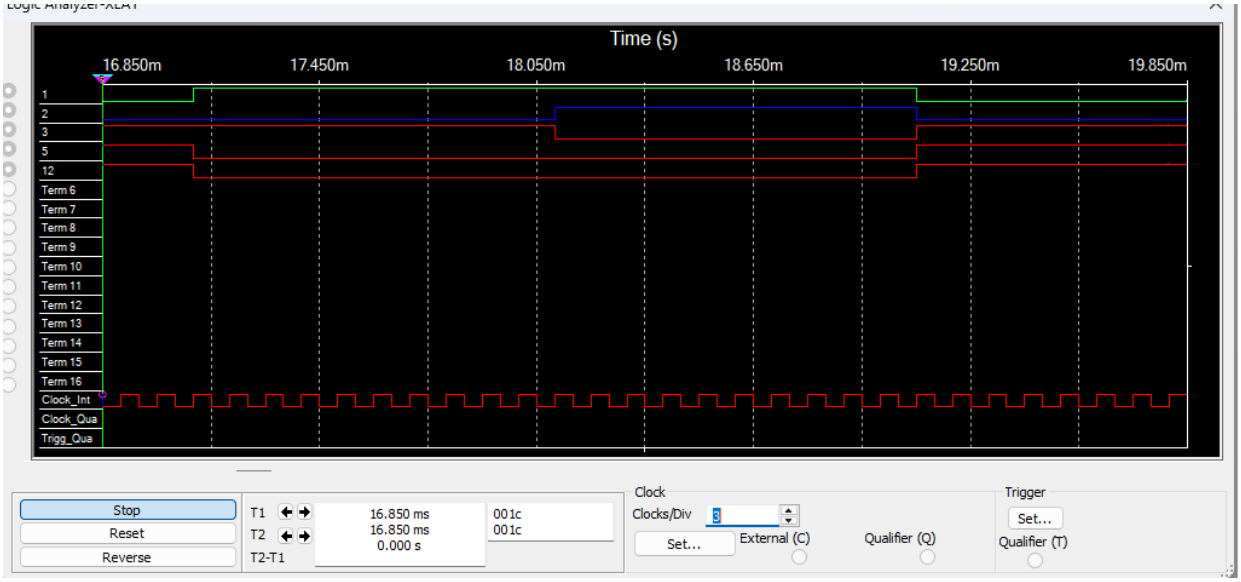

Figure 4.3: Waveform of the minterm expansion and xor equations for full subtractor

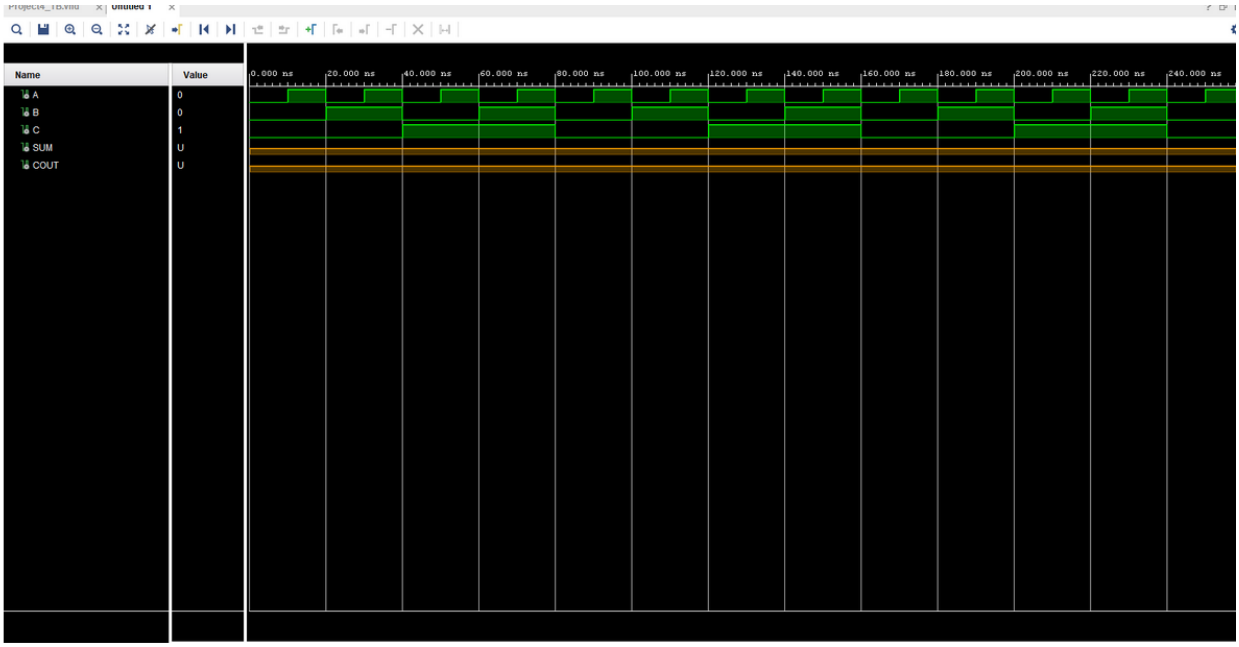

Figure 6.1: Behavioral simulation the minterm expansion and the simplified xor expression

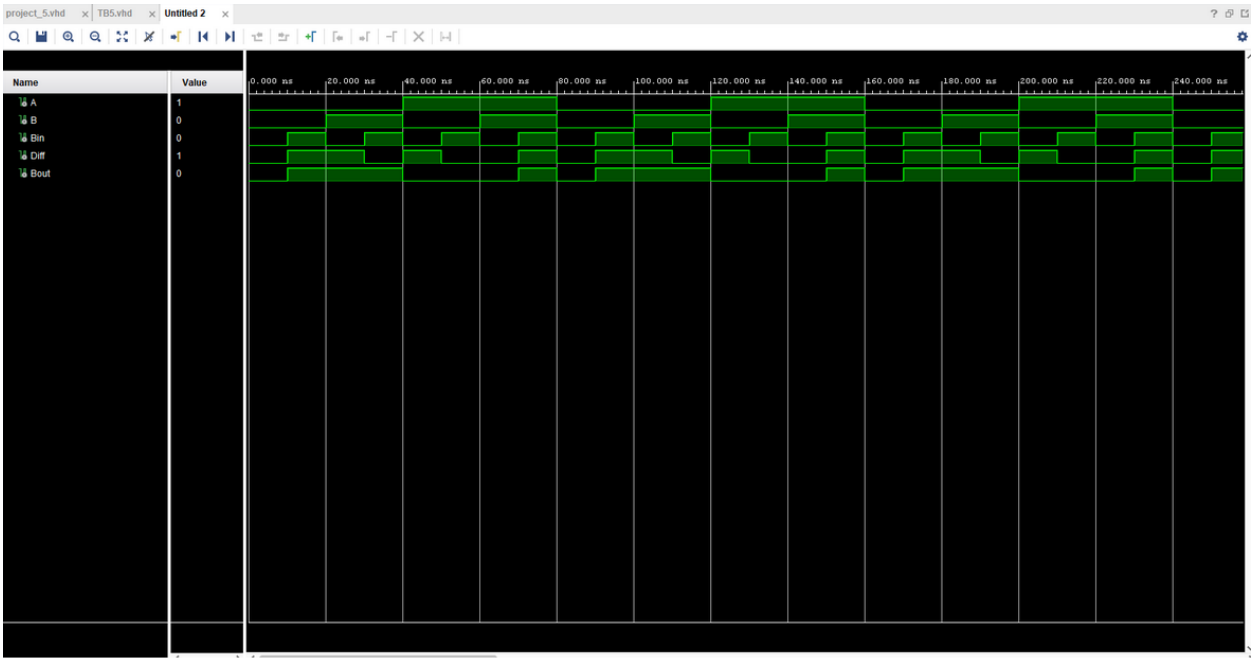

Figure 6.2: Behavioral simulation the minterm expansion and the simplified xor expression for full subtractor

The result in figure 7.1 shows the input that the program is submitting is 111 and the output is 1 with the wiring being a xor version of the minterm expression.
The result in figure 7.2 shows the input that the program is submitting is 111 and the output is 1

with the wiring being a minterm expansion of the full adder.
The result in figure 7.3 shows the input that the program is submitting is 100 and the output is 1 with the wiring being a xor version of the minterm expression.
The result in figure 7.4 shows the input that the program is submitting is 100 and the output is 1 with the wiring being a minterm expansion of the full adder
The result in figure 7.5 shows the input that the program is submitting is 010 and the output is 1 with the wiring being a xor version of the minterm expression.
The result in figure 7.6 shows the input that the program is submitting is 010 and the output is 1 with the wiring being a minterm expansion of the full adder.
The result in figure 7.7 shows the input that the program is submitting is 001 and the output is 1 with the wiring being a xor version of the minterm expression.
The result in figure 7.8 shows the input that the program is submitting is 001 and the output is 1 with the wiring being a minterm expansion of the full adder.
The result in figure 7.9 shows the input that the program is submitting is 111 with the Diff being 1 and Bout being 1 with the wiring being a minterm expansion and the xor equation of the full subtractor.
The result in figure 7.10 shows the input that the program is submitting is 001 with the Diff being 1 and Bout being 0 with the wiring being a minterm expansion and the xor equation of the full subtractor.
The result in figure 7.11 shows the input that the program is submitting is 010 with the Diff being 1 and Bout being 1 with the wiring being a minterm expansion and the xor equation of the full subtractor.
The result in figure 7.12 shows the input that the program is submitting is 001 with the Diff being 1 and Bout being 0 with the wiring being a minterm expansion and the xor equation of the full subtractor.

The result in figure 7.13 shows the Arduino board of a full adder with an input of 011 will produce a sum of 1 and a carry of 1.

The result in figure 7.14 shows the Arduino board of a full adder with an input of 110 will produce a sum of 0 and a carry of 1.

The result in figure 7.15 shows the Arduino board of a full adder with an input of 100 will produce a sum of 1 and a carry of 0.
The result in figure 7.16 shows the Arduino board with of a full subtractor an input of 100 will produce a Diff of 1 and a Bout of 0.

The result in figure 7.17 shows the Arduino board with of a full subtractor an input of 111 will produce a Diff of 1 and a Bout of 1.

The result in figure 7.18 shows the Arduino board with of a full subtractor an input of 010 will produce a Diff of 1 and a Bout of 1.

The result in figure 7.19 shows the Arduino board with of a full subtractor an input of 001 will produce a Diff of 1 and a Bout of 1.

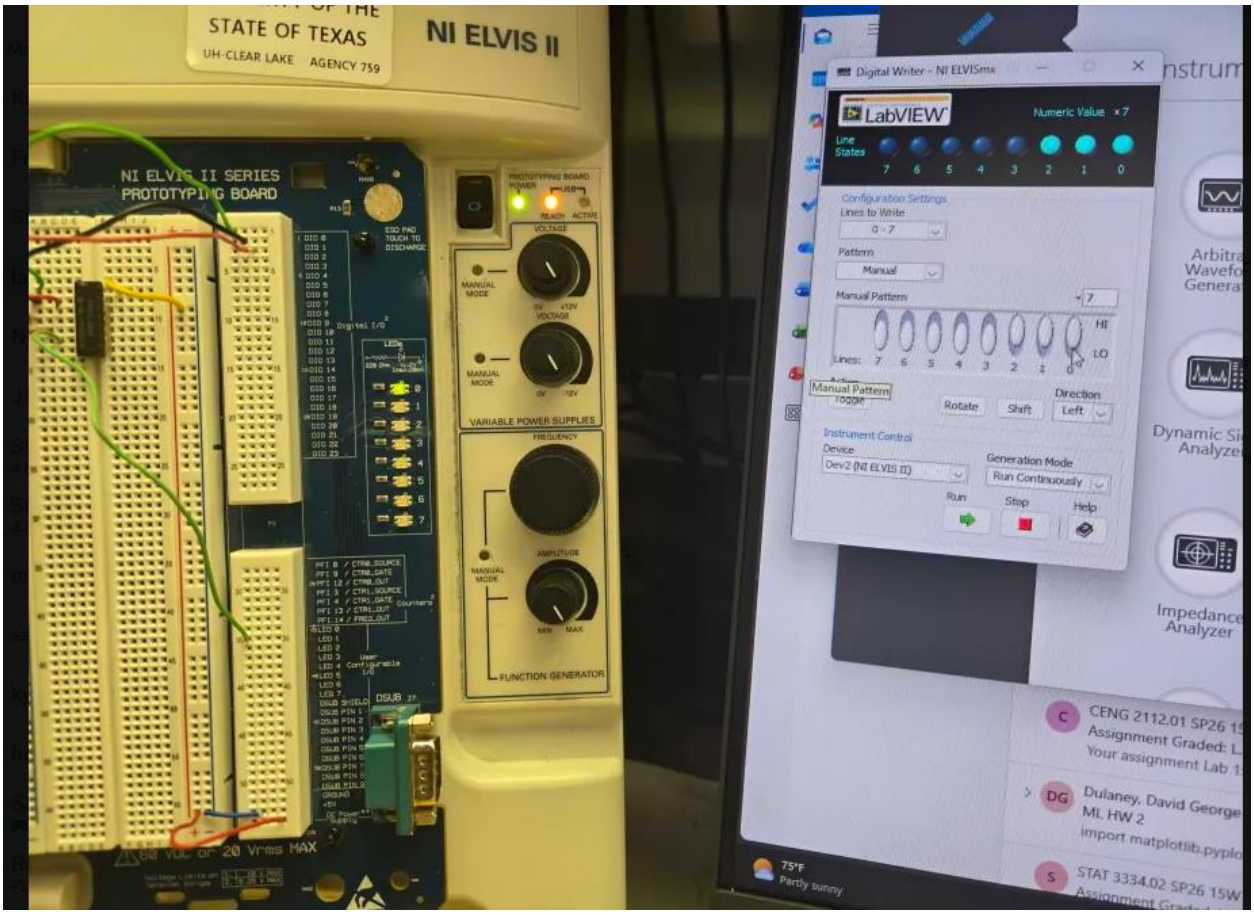

Figure 7.1: physical board showing the xor expression for full adder with an input of the of the bits from LSB to MSB being 111

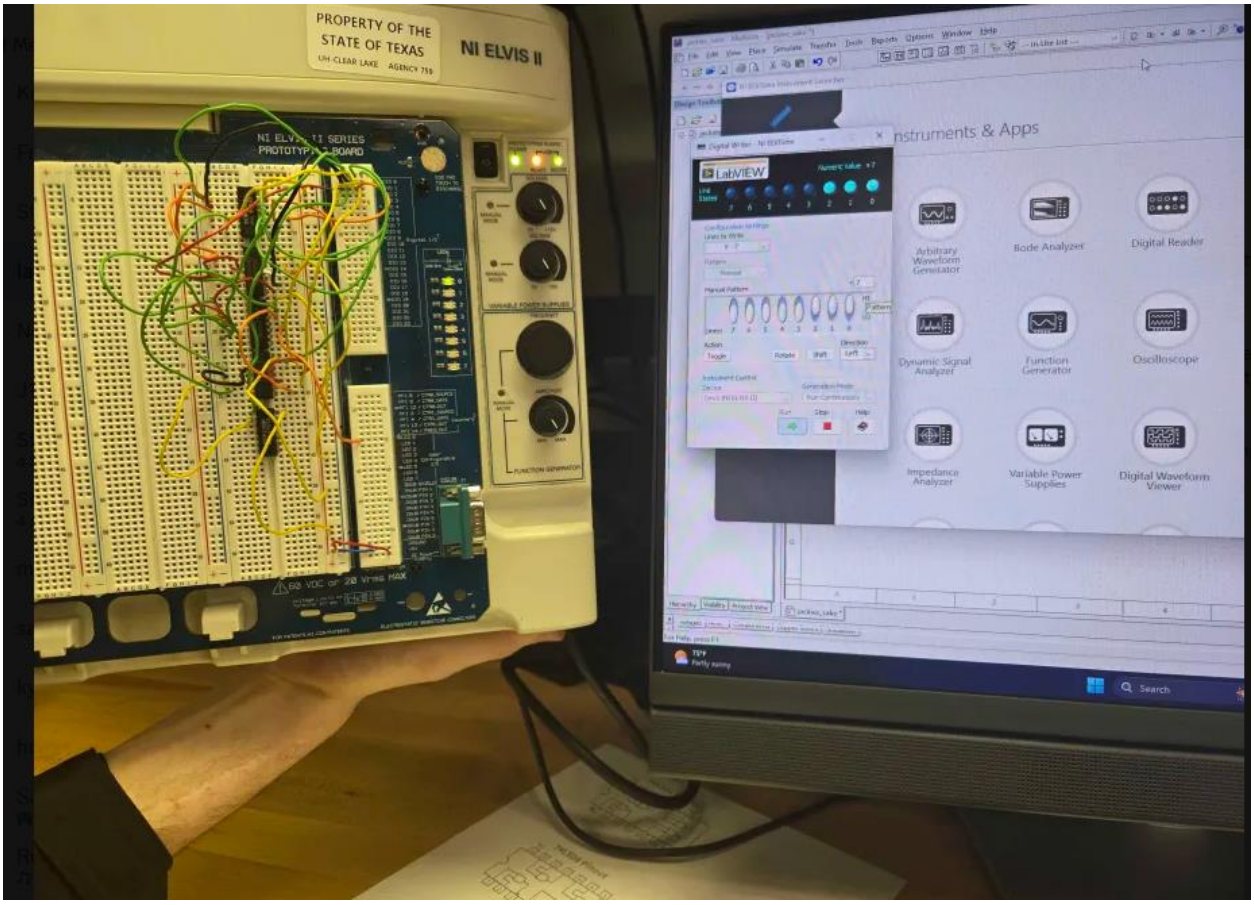

Figure 7.2: physical board showing the minterm expansion for full adder with an input of the of the bits from LSB to MSB being 111

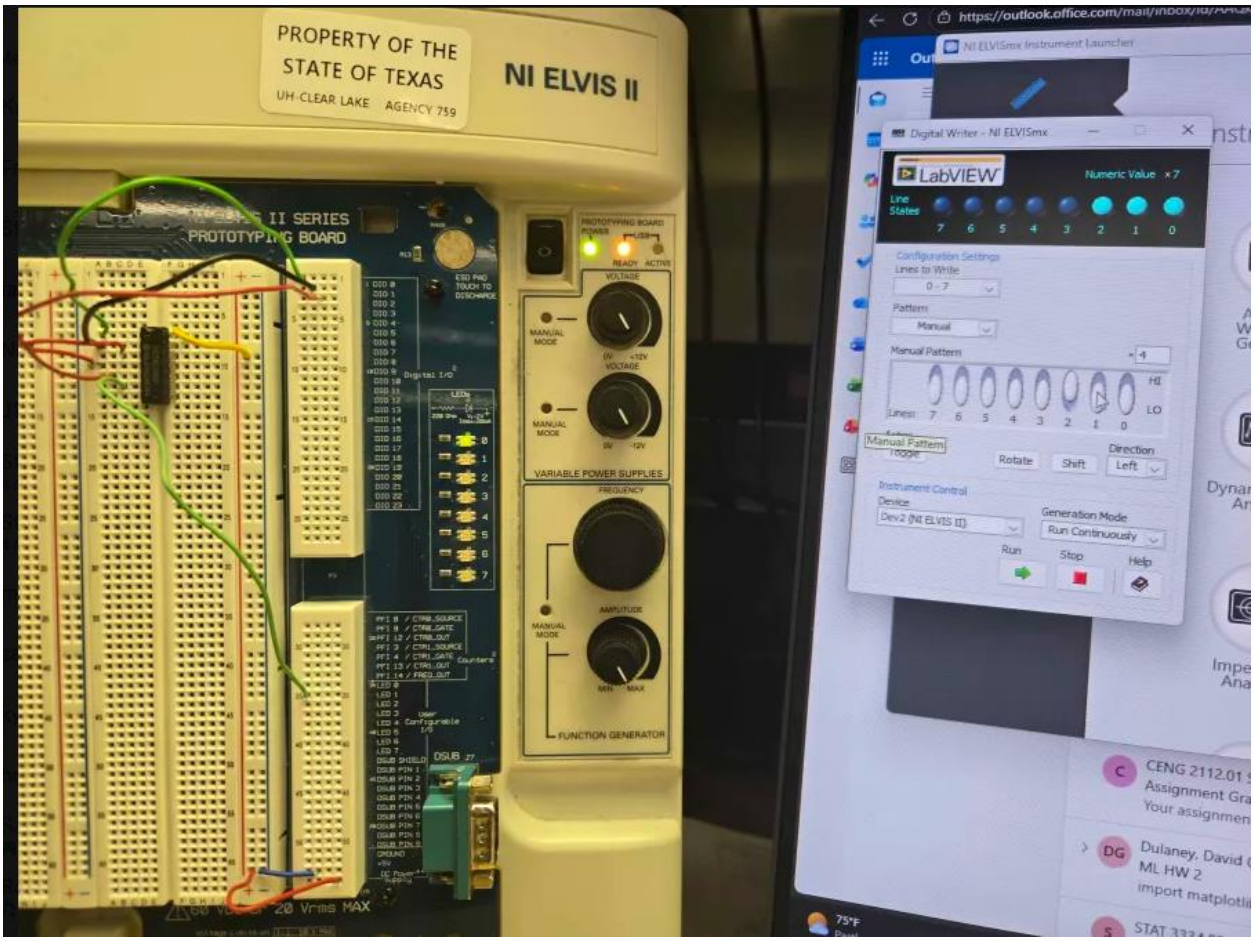

Figure 7.3: physical board showing the xor expression for full adder with an input of the of the bits from LSB to MSB being 100

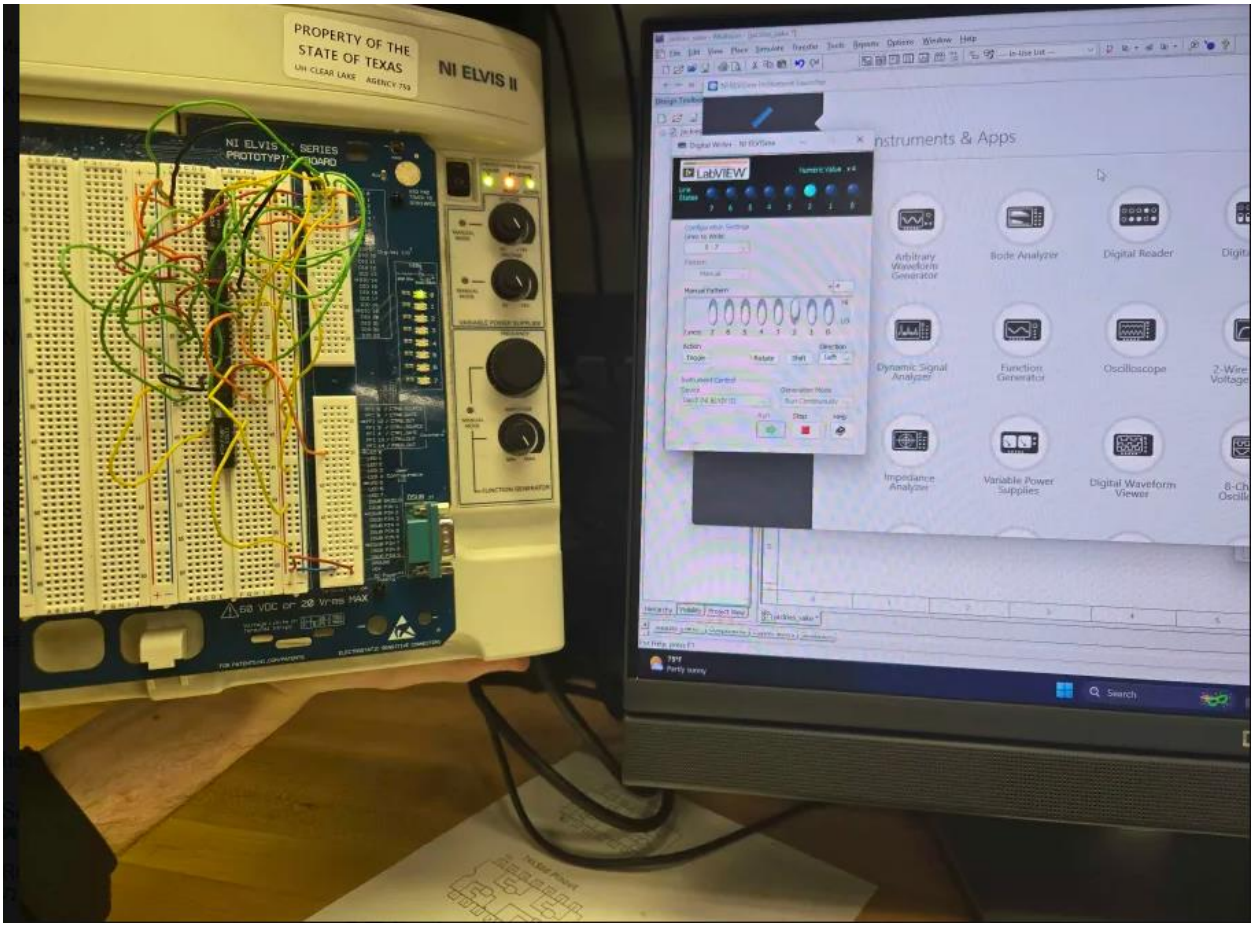

Figure 7.4: physical board showing the minterm expansion for full adder with an input of the of the bits from LSB to MSB being 100

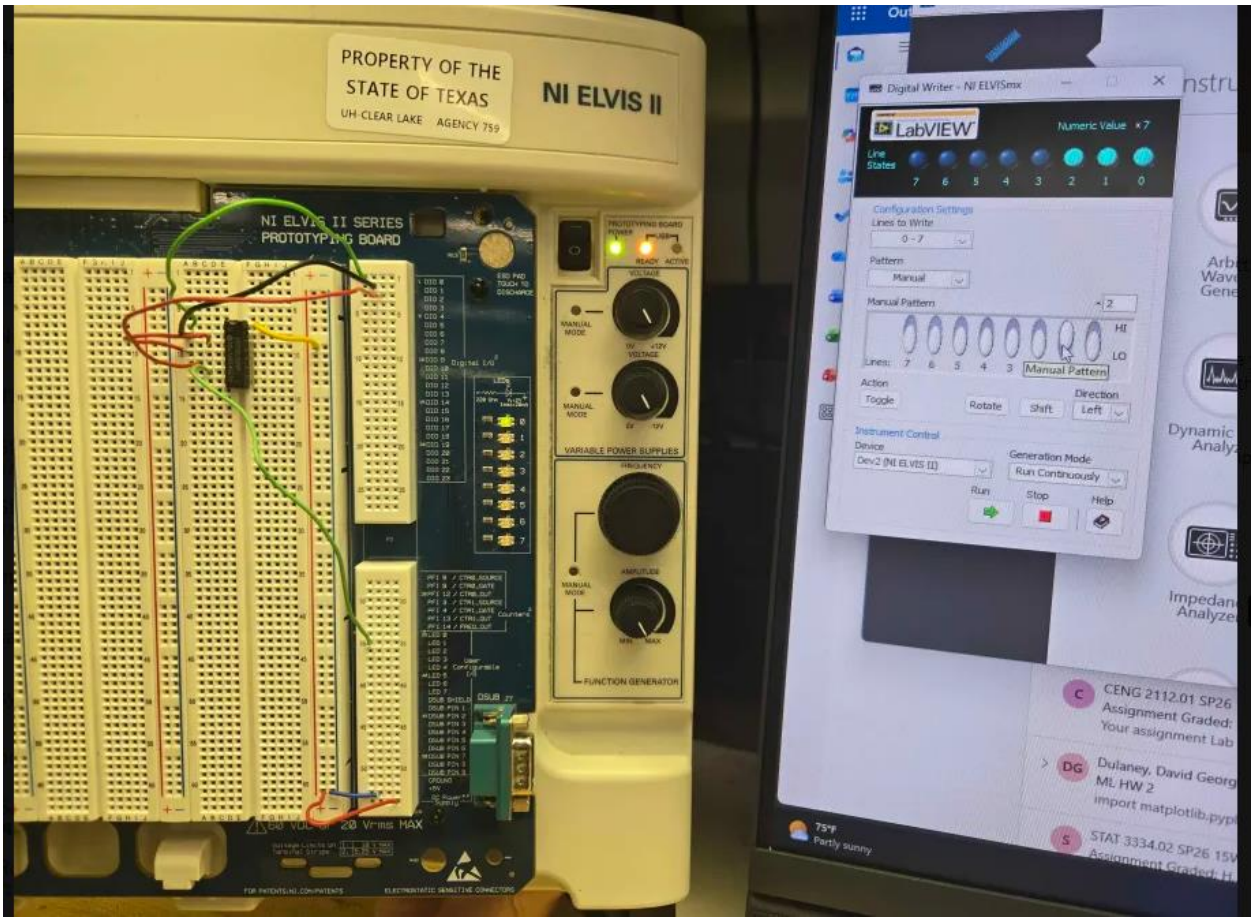

Figure 7.5: physical board showing the xor expression for full adder with an input of the of the bits from LSB to MSB being 010

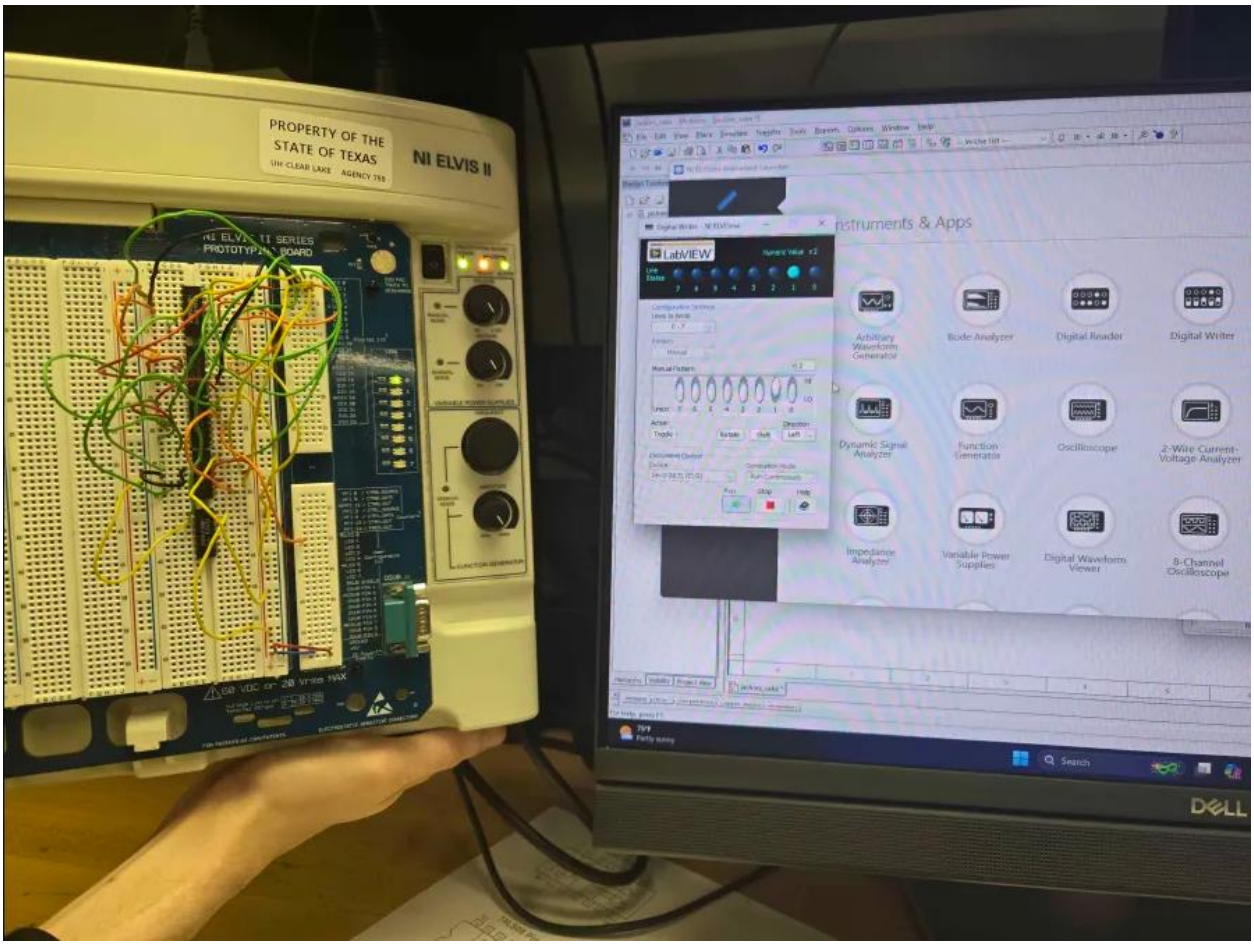

Figure 7.6: physical board showing the minterm expansion for full adder with an input of the of the bits from LSB to MSB being 010

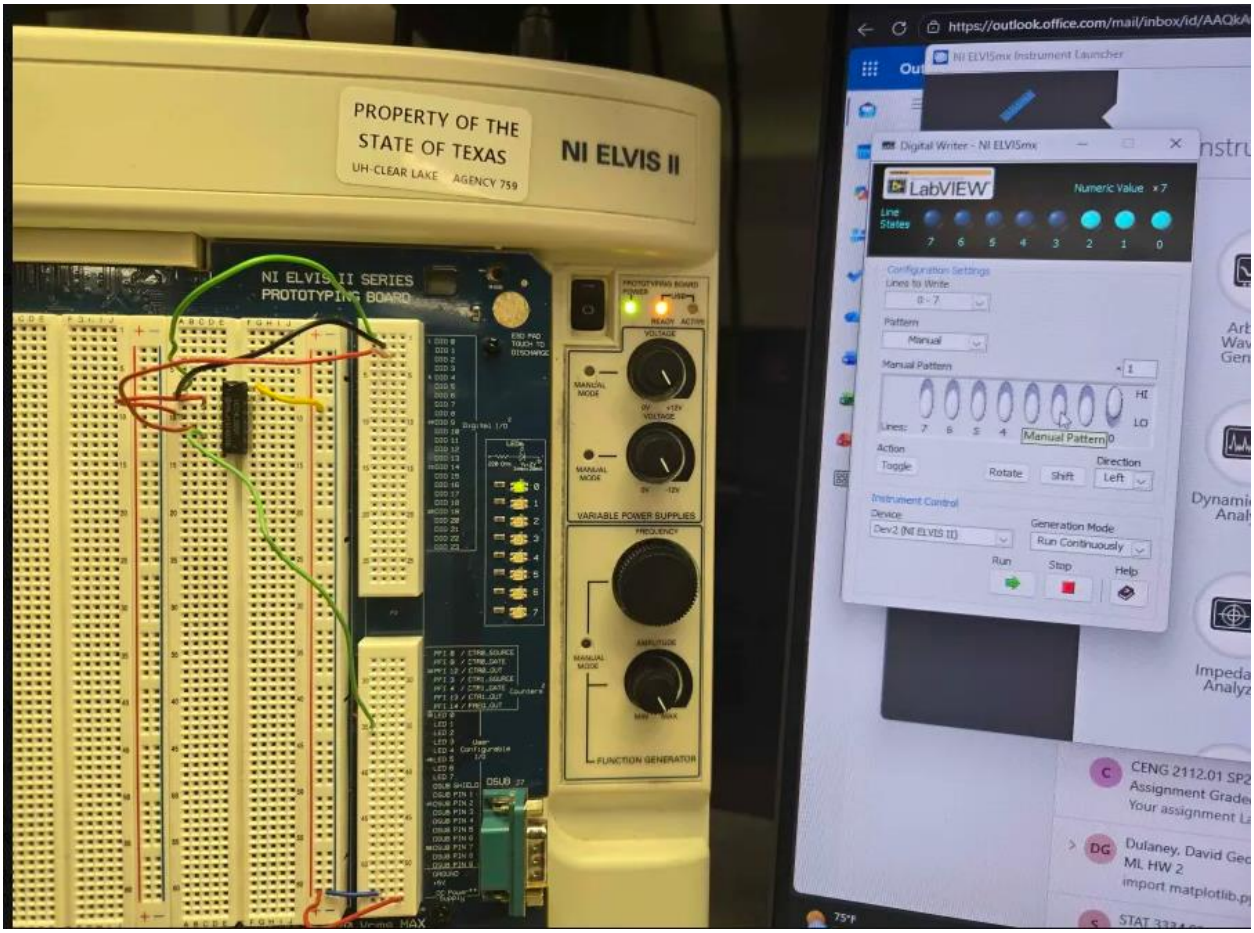

Figure 7.7: physical board showing the xor expression for full adder with an input of the of the bits from LSB to MSB being 001

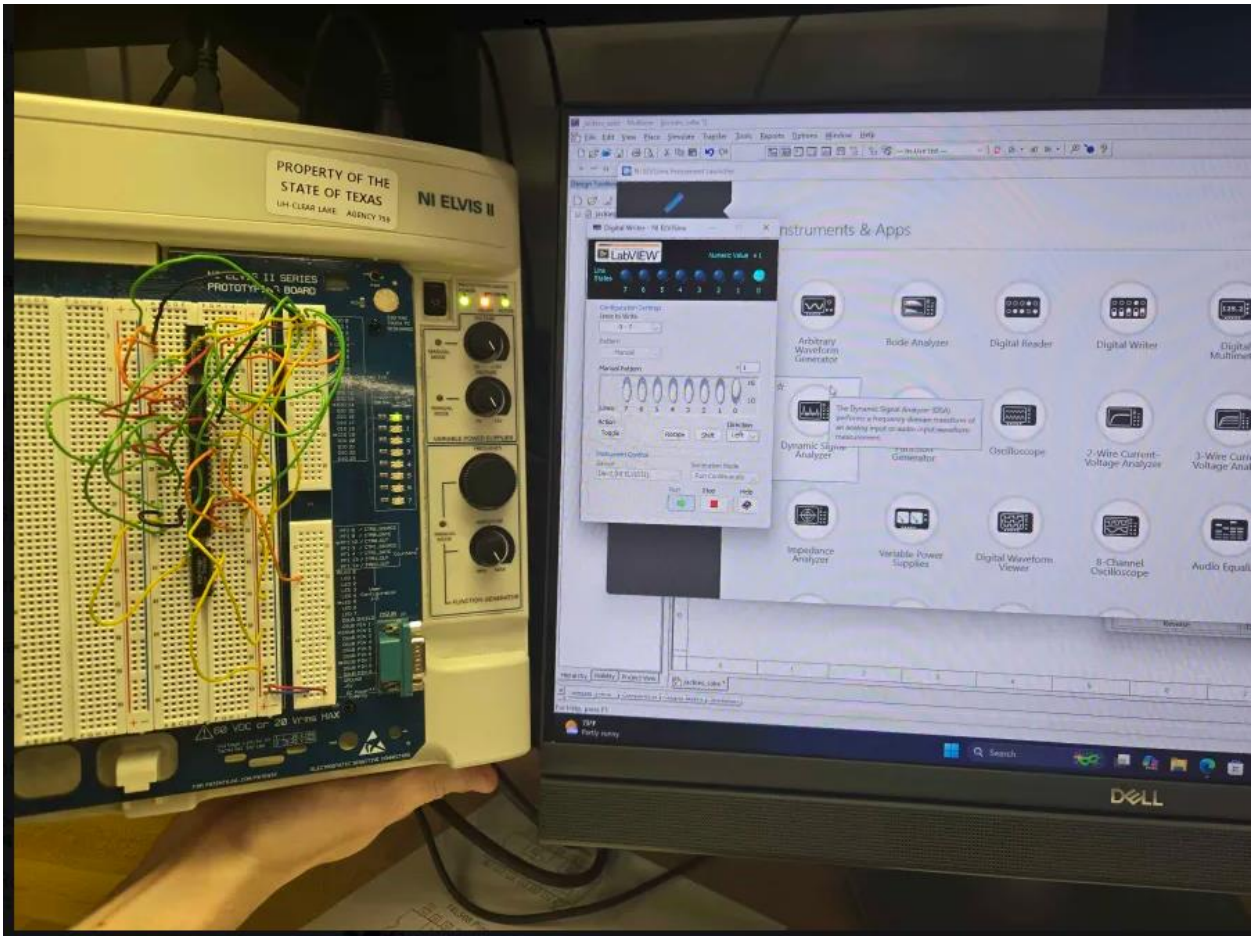

Figure 7.8: physical board showing the minterm expansion for full adder with an input of the of the bits from LSB to MSB being 001

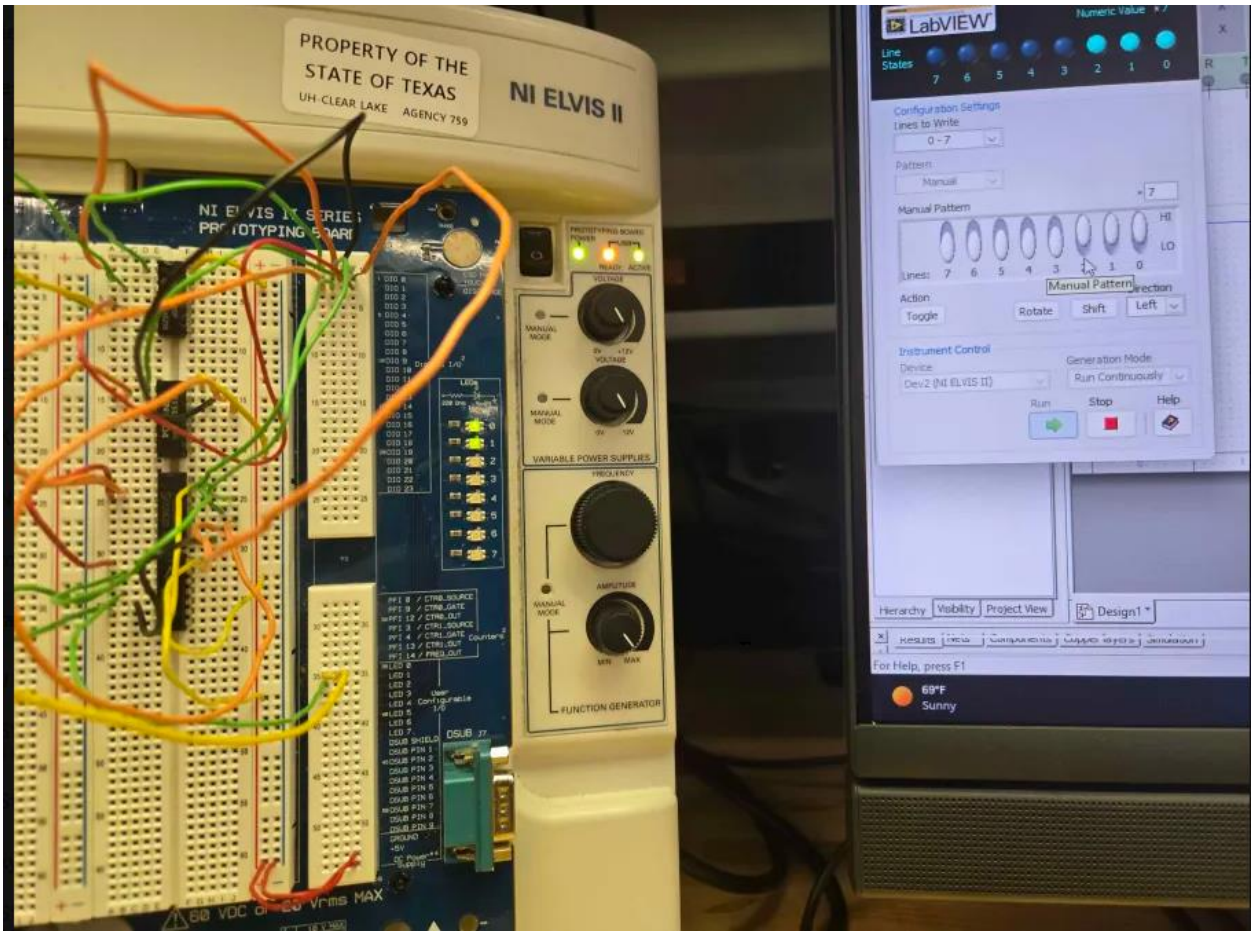

Figure 7.9: physical board showing the minterm expansion and xor equation for full subtractor with an input of the of the bits from LSB to MSB being 111

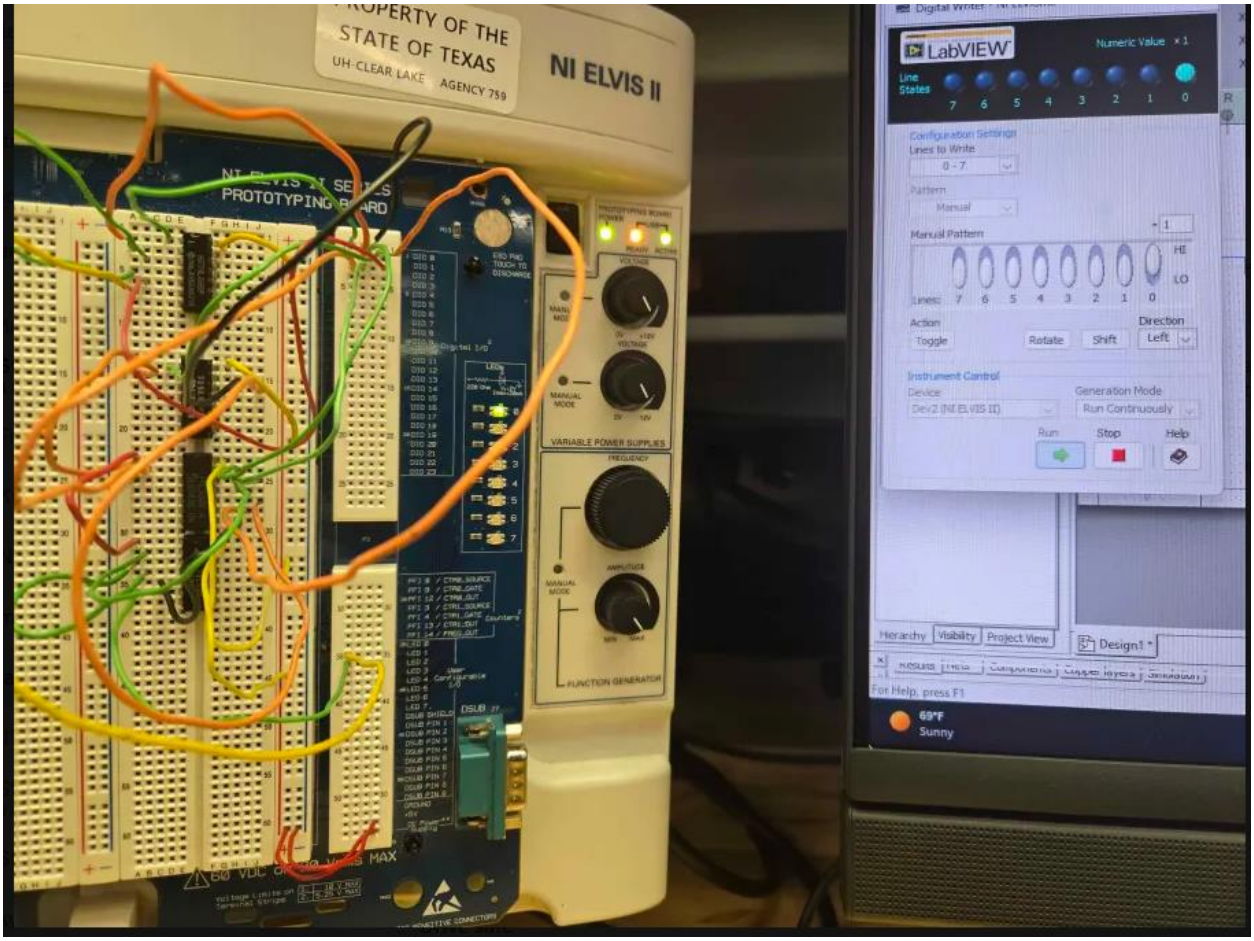

Figure 7.10: physical board showing the minterm expansion and xor equation for full subtractor with an input of the of the bits from LSB to MSB being 001

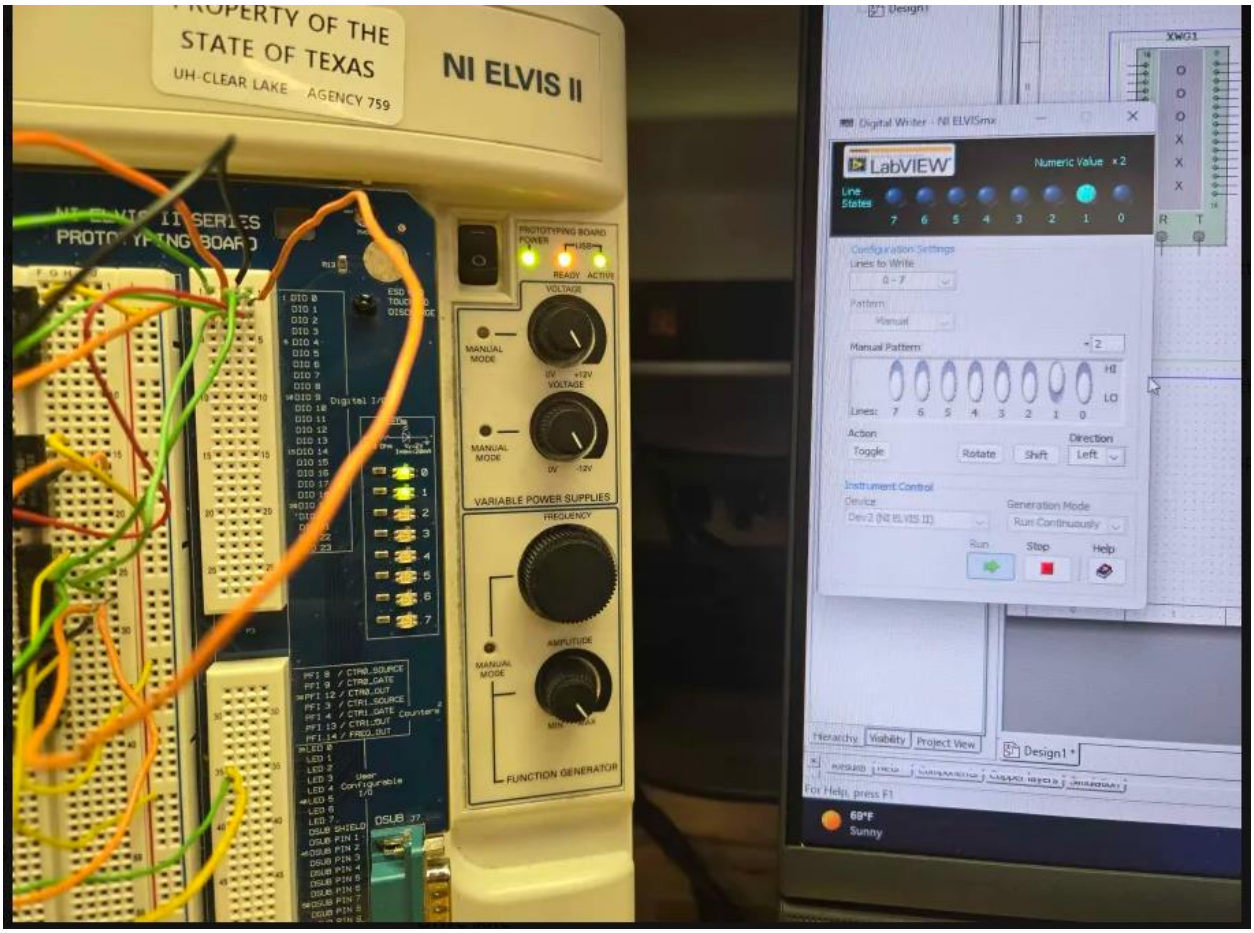

Figure 7.11 physical board showing the minterm expansion and xor equation for full subtractor with an input of the of the bits from LSB to MSB being 010

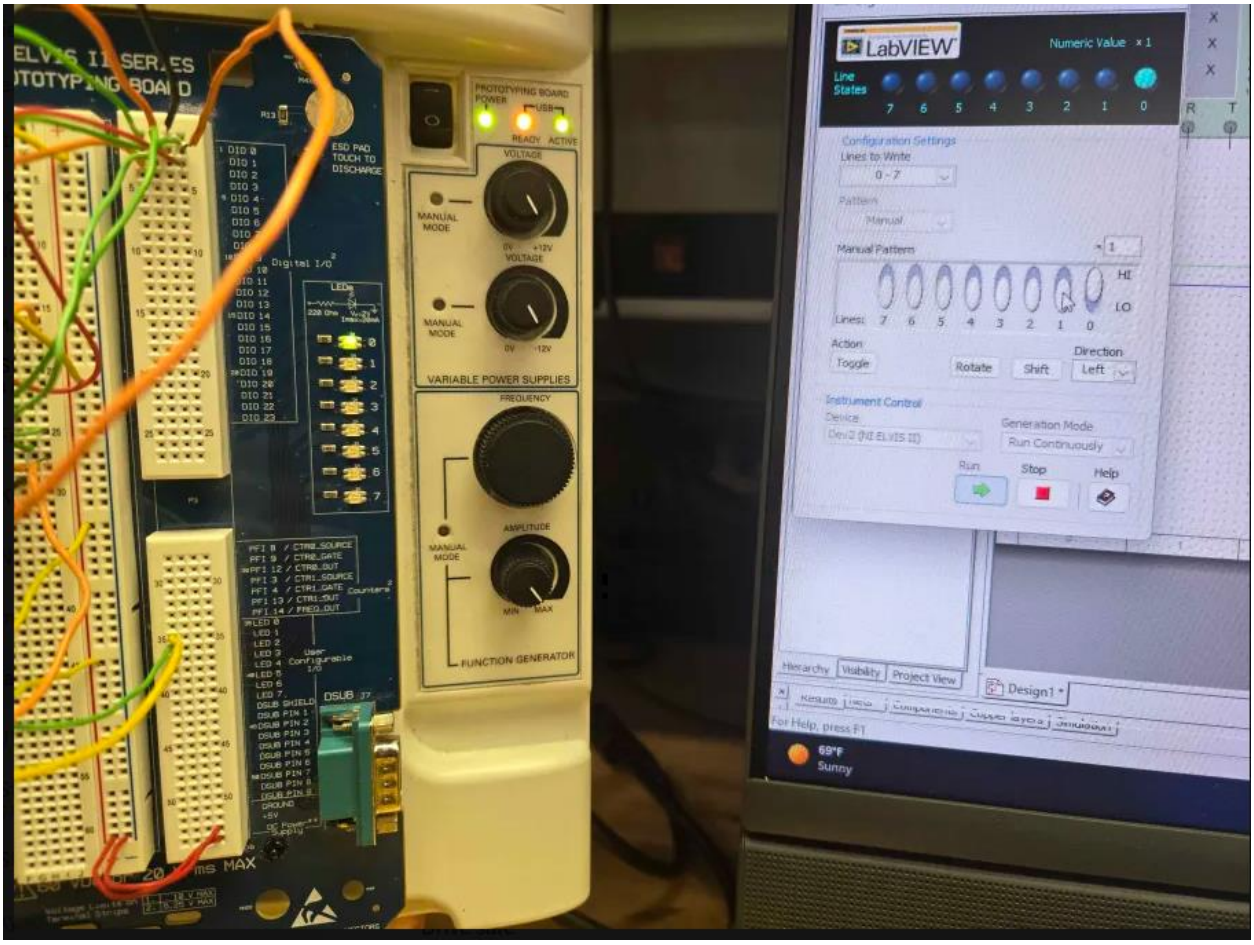

Figure 7.12 physical board showing the minterm expansion and xor equation for full subtractor with an input of the of the bits from LSB to MSB being 001

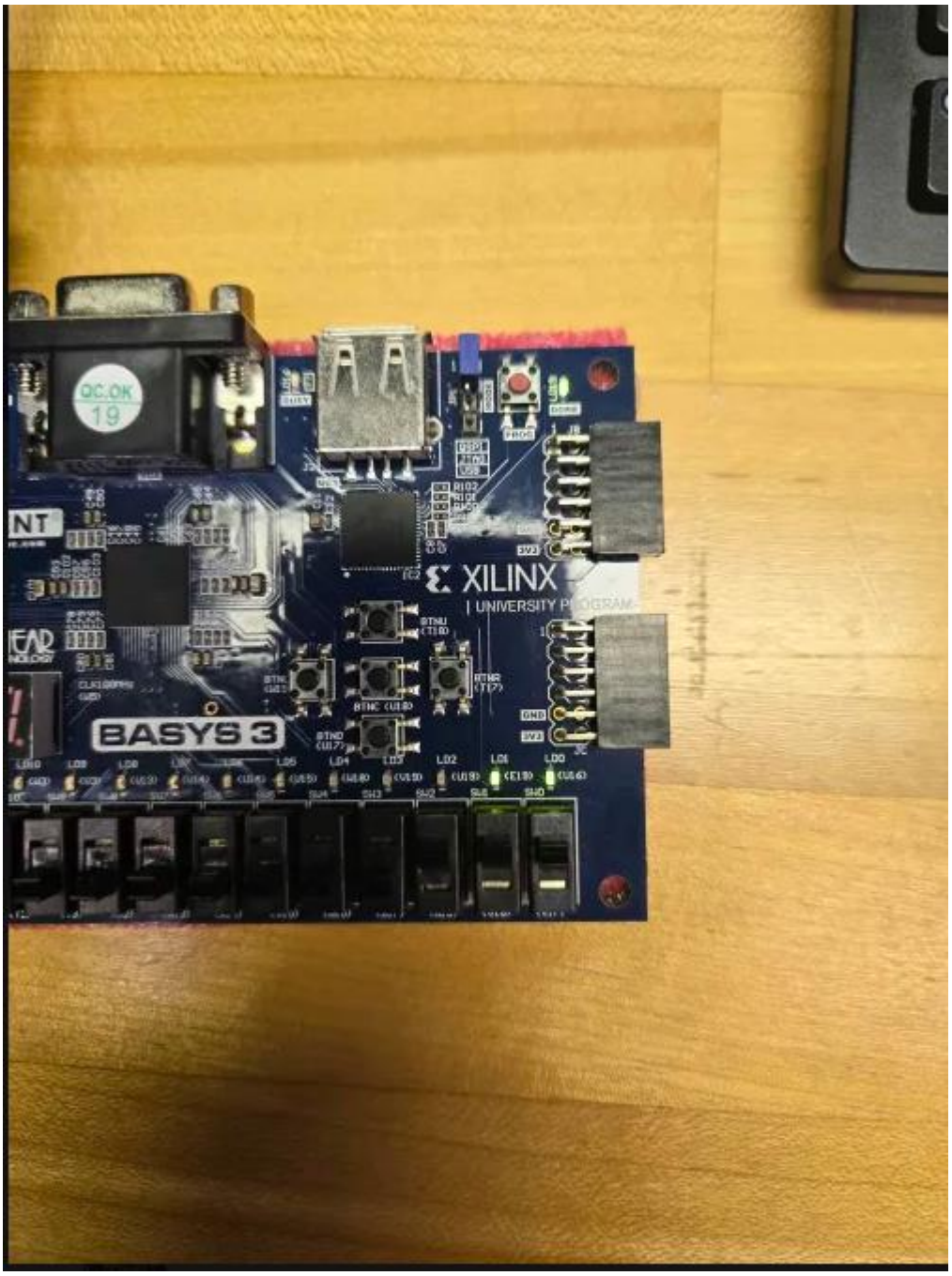

Figure 7.13: Arduino board for full adder with an input of the bits from LSB to MSB being 011

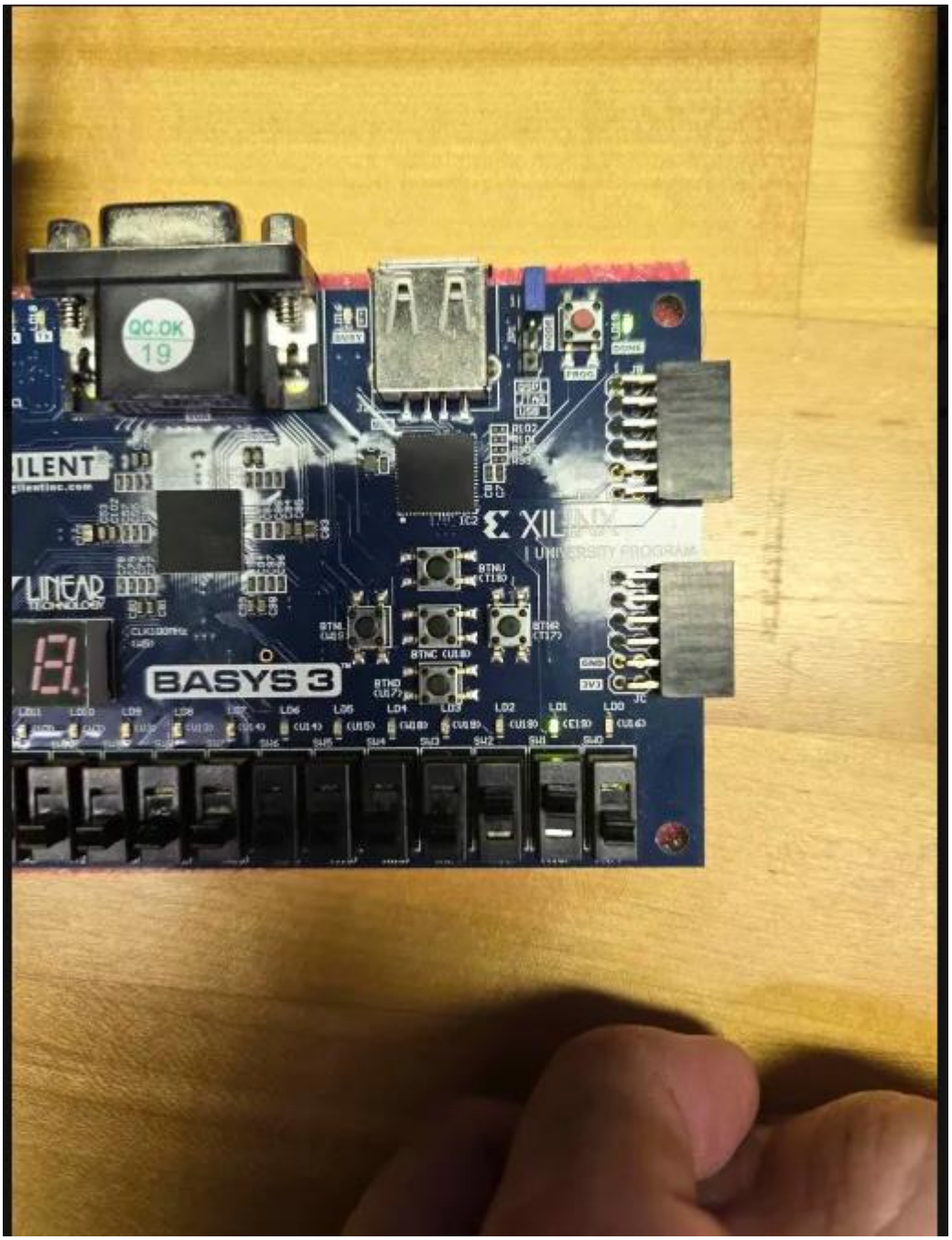

Figure 7.14: Arduino board for full adder with an input of the bits from LSB to MSB being 010

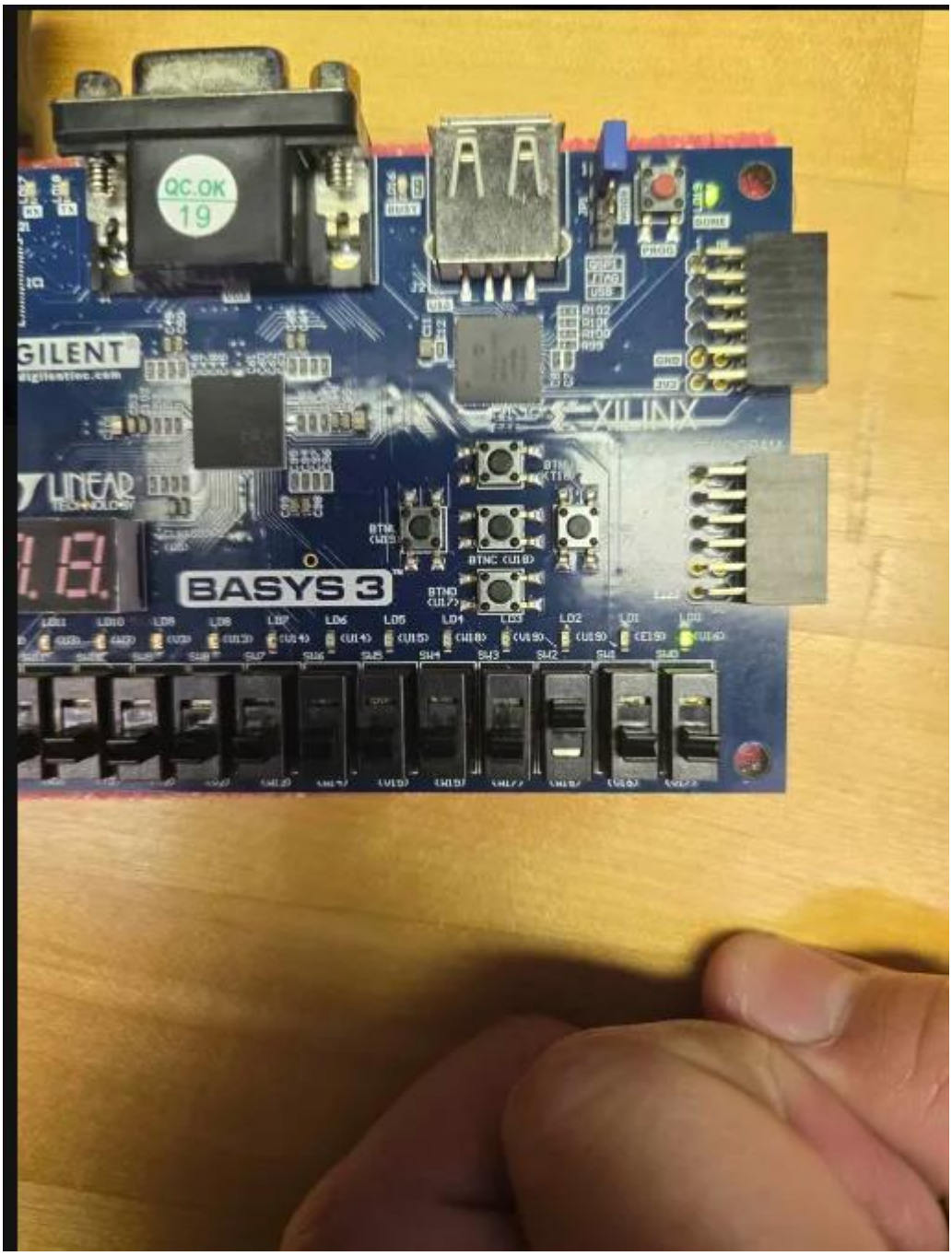

Figure 7.15: Arduino board for full adder with an input of the bits from LSB to MSB being 100

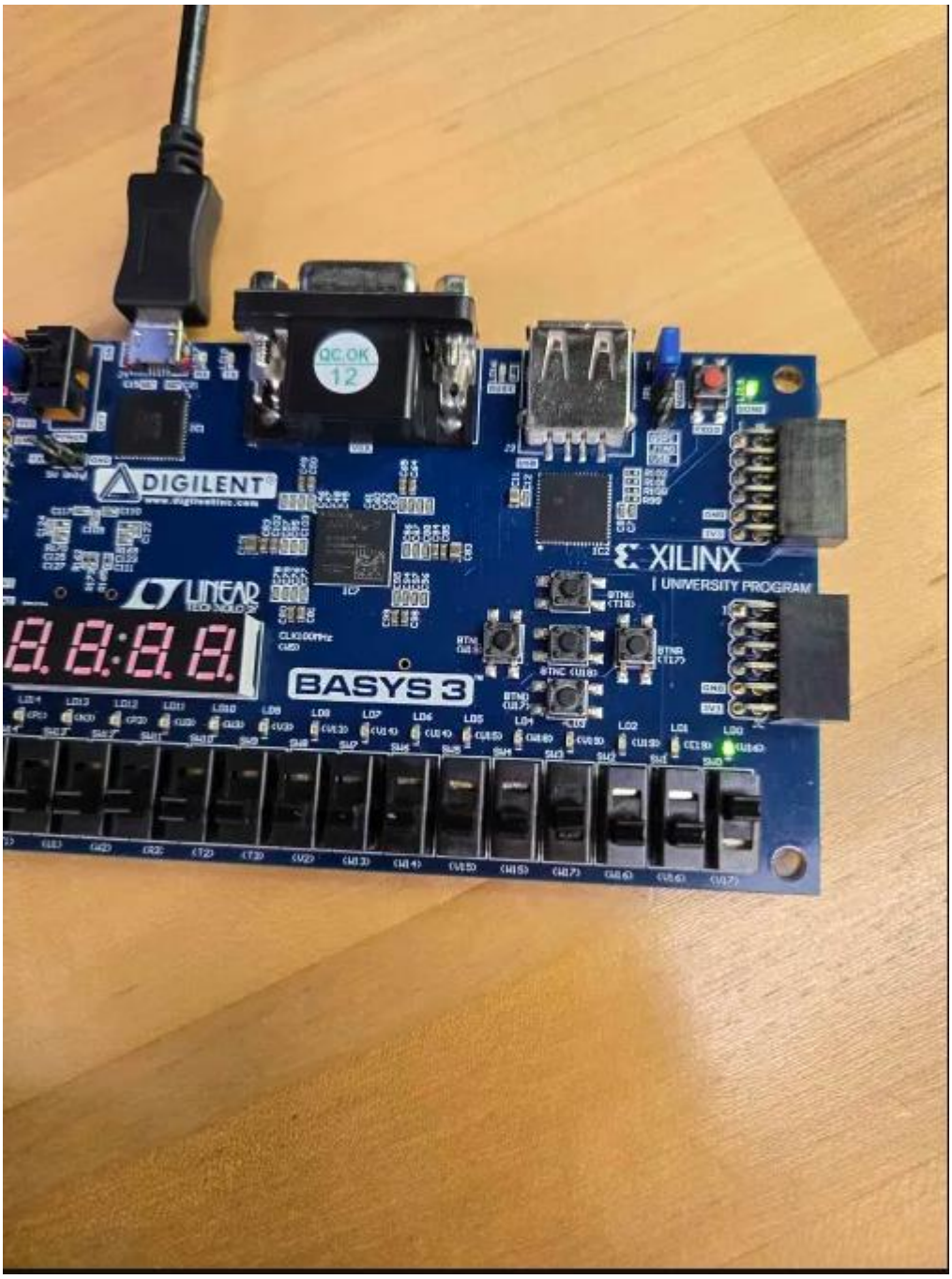

Figure 7.16: Arduino board for full subtractor with an input of the bits from LSB to MSB being 100

Figure 7.17: Arduino board for full subtractor with an input of the bits from LSB to MSB being 111

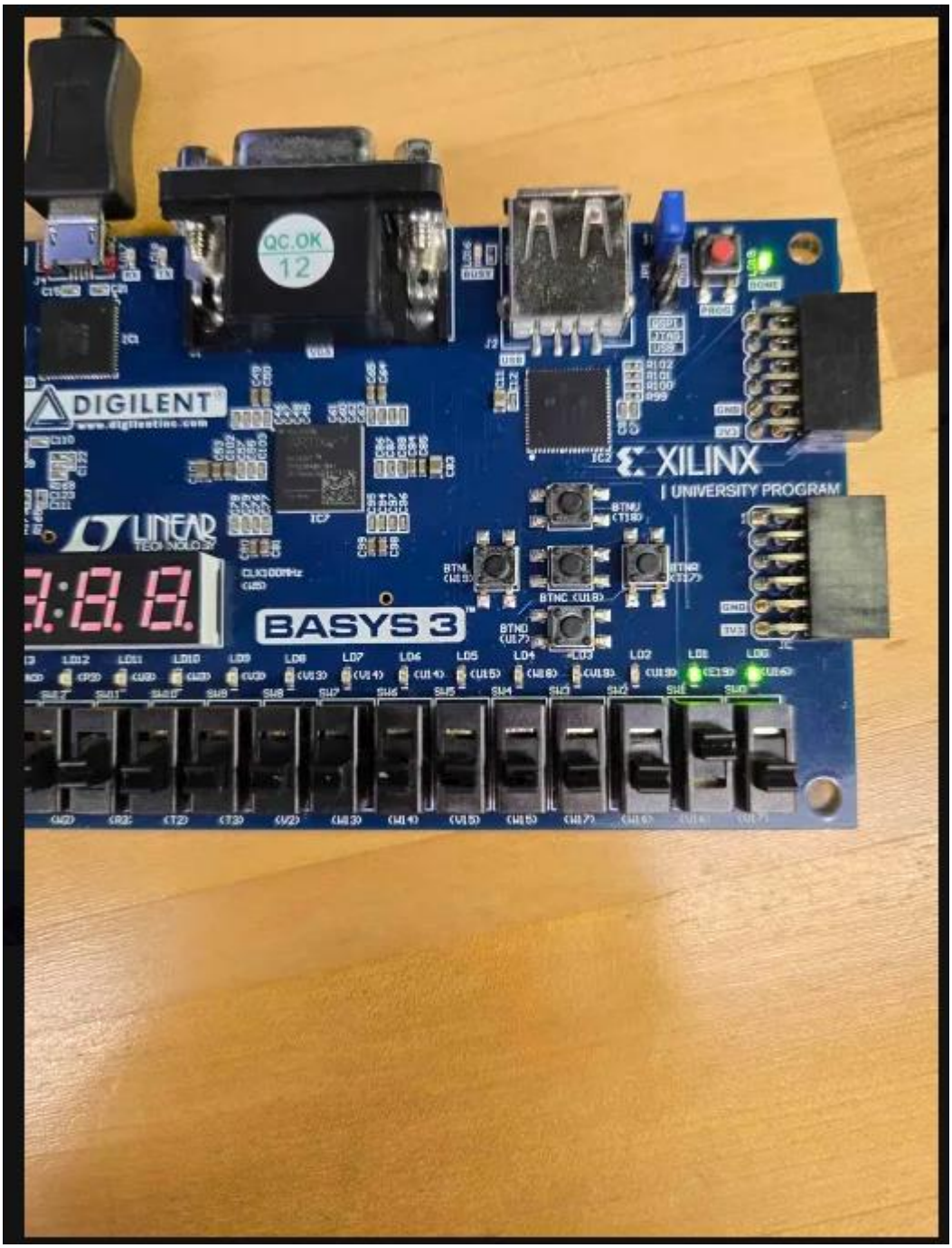

Figure 7.18: Arduino board for full subtractor with an input of the bits from LSB to MSB being 010

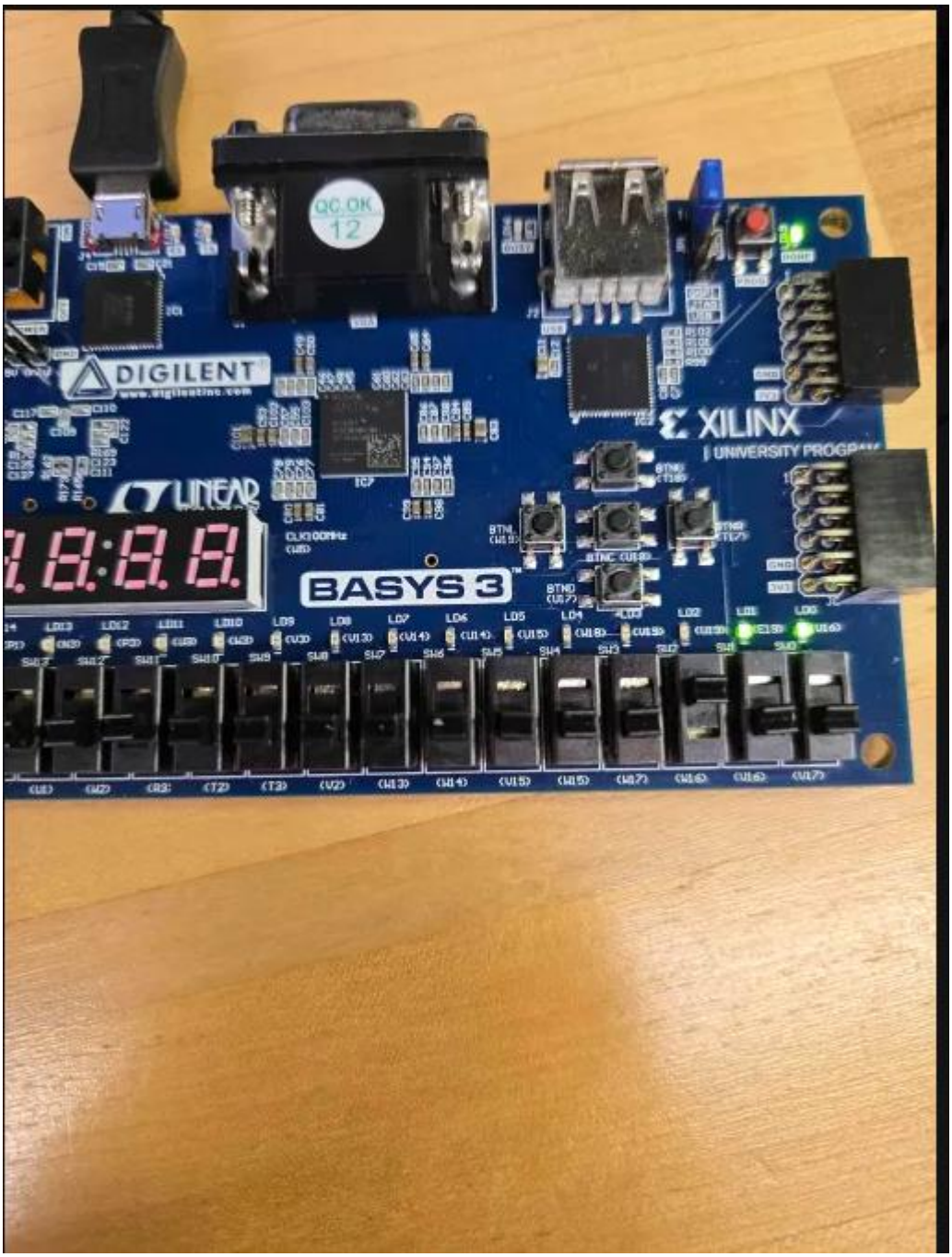

Figure 7.19: Arduino board for full subtractor with an input of the bits from LSB to MSB being 001

Conclusion

This project was designed to understand that the minterm expansion of a full adder and full subtractor along with the simplified version of the xor expression. The main takeaway is that

finding the minterm expansion can help open a way to simplify a circuit using xor gates. This optimizes the amount of gates needed to produce the same output as a minterm expansion which is much longer and expensive to use.